

Unidade III

5 PROJETO E CONSTRUÇÃO DO SOFTWARE

O projeto e a construção do software representam o momento em que a visão conceitual se transforma em produto funcional, exigindo sinergia entre planejamento, modelagem, arquitetura e codificação orientada à qualidade.

No desenvolvimento do projeto de software, são estruturados diversos processos sistemáticos de acompanhamento e monitoramento que abrangem todas as suas fases da concepção, implementação e implantação, incluindo suporte e manutenção.

5.1 Gerenciamento do projeto de software

O ciclo de vida do projeto é gerenciado através da execução de uma série de atividades conhecidas como **processo de gerenciamento de projetos**.

Um processo é um conjunto de atividades, ações e tarefas realizadas na criação de algum artefato. Uma atividade se esforça para atingir um objetivo amplo (p. ex., comunicar-se com os envolvidos) e é utilizada independentemente do domínio de aplicação, do tamanho do projeto, da complexidade dos esforços ou do grau de rigor com que a engenharia de software será aplicada. Uma ação (p. ex., projeto de arquitetura) envolve um conjunto de tarefas que resultam em um artefato de software fundamental (p. ex., um modelo arquitetural). Uma tarefa se concentra em um objetivo pequeno, porém bem definido (p. ex., realizar um teste de unidades), e produzir um resultado tangível (Pressman; Maxim, 2021).

Seguindo os princípios estabelecidos no Guia PMBOK® (2017), todo processo deve ser claramente delimitado, definindo explicitamente seu ponto de início (input) e seu ponto de término (output). A figura 25, a seguir, ilustra essas delimitações, destacando os marcos de referência (milestones) que representam eventos significativos no ciclo de vida do projeto.

Esses marcos são fundamentais para o monitoramento e controle do processo, permitindo medir o progresso, avaliar o desempenho, identificar desvios e projetar melhorias e tendências de forma contínua, assegurando a conformidade com o plano de gerenciamento do projeto.



Figura 25 – Modelo básico do processo

- **Input (entradas):** representa o ponto inicial do processo, correspondendo aos requisitos e aos recursos necessários para iniciar uma atividade ou ciclo de desenvolvimento. Esses itens, em uma forma ampla, referem-se aos requisitos de entrada do processo, tais como: objetos claros e específicos, recursos humanos necessários, materiais, tecnologia a ser empregada, fornecedores, componentes do software/sistema, metodologias, ferramentas, custo, cronograma, procedimentos de trabalho e documentos.

Em um contexto ágil, os inputs incluem: histórias de usuário, backlog priorizado, recursos humanos e tecnológicos, infraestrutura de desenvolvimento, ferramentas de integração e automação, padrões de qualidade, métricas de desempenho, estimativas de custo e tempo, além de documentação técnica essencial. Esses elementos constituem a base sobre a qual o processo será planejado e executado.

- **Processo (atividades e tarefas):** é a combinação estruturada e iterativa que organiza os itens de entrada para criar um produto de software ou de sistema, ou um componente de outro produto, ou para executar um determinado trabalho. Por meio de práticas ágeis e ferramentas colaborativas, o processo envolve a transformação contínua dos requisitos em incrementos de software funcional, a partir de sprints, integração e entrega contínuas, testes automatizados, revisões de código e feedback constante do cliente. Dessa forma, o processo torna-se dinâmico, adaptativo e centrado na geração de valor.
- **Output (saídas):** são os resultados finais gerados pelo processo. Corresponde à entrega validada do produto, componente ou serviço, conforme os critérios de aceitação definidos no processo. Novos registros são feitos e novas medições e verificações são realizadas para confrontar o resultado obtido com o objetivo planejado para o processo. No encerramento do processo, são realizadas medições, verificações e registros que comparam os resultados obtidos com os objetivos planejados, promovendo o aprendizado organizacional e a melhoria contínua. Em ambientes ágeis, o output também alimenta novos ciclos de desenvolvimento, garantindo a retroalimentação e a evolução incremental do produto.

Em cada processo, o engenheiro de software se depara com um conjunto de alternativas de componentes, métodos de integração, frameworks e ferramentas tecnológicas, que precisam ser cuidadosamente avaliadas quanto à sua adequação técnica e custo-benefício.

Veja o quadro a seguir. Uma vez adotado componentes, métodos, técnicas e ferramentas, a equipe constrói um framework de uso do seu **arcabouço do processo**. É como uma "caixa de ferramentas" com recursos para projetar e construir software, em que a equipe desenvolvedora se baseia para organizar as atividades, definir práticas recomendadas e orientar o uso de tecnologias de apoio. Esse arcabouço estabelece a base metodológica sobre a qual se estruturam as fases de planejamento, design, implementação e verificação, permitindo que o desenvolvimento ocorra de forma adaptativa e incremental.

Quadro 13 – Arcabouço do processo com métodos, ferramentas e técnicas (MF&T)

Métodos	Ferramentas	Técnicas
Cascata	Astah	
Incremental	Azure DevOps	
Espiral	CASE	Análise de documentos
Scrum	Jira	User stories
XP	Trello	Prototipagem
Kanban	Figma	Modelagem UML
FDD	Android Studio	Arquiteturas
SAFe®	Eclipse	Padrões de projeto
DevOps	NetBeans	Técnicas de refatoração
SOA	PyCharm	Testes
ISO 12207	Visual Studio	Qualidade, PDCA
ISO 25000	GitHub	Matriz SWOT
CMMI®	Microsoft Teams	Versionamento
MPS.BR	Microsoft Project	
	Google Cloud Platform	

A definição do arcabouço segue um fluxo iterativo em que, durante a análise do contexto do projeto, são filtradas as alternativas de métodos, técnicas e ferramentas menos relevantes, mantendo-se apenas aquelas que se mostram essenciais para alcançar os objetivos do sistema. A etapa de abstração e modelagem do arcabouço permite representar com precisão os componentes metodológicos e os comportamentos esperados do processo, resultando em uma estrutura consistente, alinhada à qualidade do produto de software e à efetividade da engenharia de desenvolvimento.

Esse conjunto de decisões constitui o núcleo da atividade de projeto, que requer análise criteriosa e documentação detalhada para sintetizar e otimizar as atividades de desenvolvimento. O processo de projeto caracteriza-se, portanto, por uma redução progressiva das alternativas, culminando na definição de uma solução arquitetônica e funcional estável para o projeto final do software, como mostra a figura 26 a seguir.

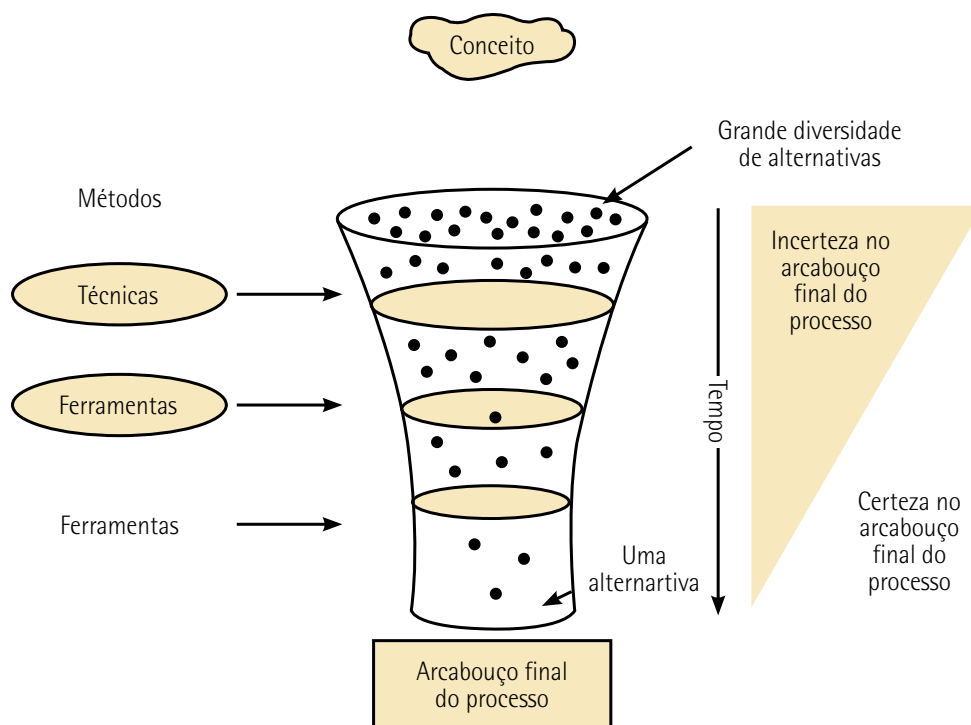


Figura 26 – Iteração para seleção de métodos, ferramentas e técnicas do arcabouço

Adaptada de: Slack (2006).

A abstração e modelagem do arcabouço não apenas formalizam os recursos essenciais do processo, como também promovem a clareza na definição dos comportamentos esperados, assegurando consistência e coerência entre as etapas de desenvolvimento. Assim, o arcabouço final emerge como um guia estruturado e confiável, capaz de orientar a execução do projeto com precisão, adaptabilidade e foco na entrega de valor. Os elementos desse arcabouço devem ser comuns aos desenvolvedores e de preferência sem versões e modelos diferentes.

5.2 Guia do conhecimento em gerenciamento de projetos PMBOK®

Um **modelo de projeto de sistema de software** deve fornecer uma visão abrangente dos elementos estruturais e funcionais que compõem o produto, incluindo estruturas de dados, arquitetura, interfaces, componentes e fluxos de integração necessários para sua implementação. Esse modelo orienta o trabalho técnico e organizacional, permitindo que as equipes planejem, executem, monitorem e entreguem soluções com qualidade, previsibilidade e valor agregado.

O **Guia PMBOK® – Project Management Body of Knowledge** (Guia do Conhecimento em Gerenciamento de Projetos) representa o principal modelo de referência internacional, justamente, para o Gerenciamento de Projetos. Embora suas origens estejam fortemente ligadas ao desenvolvimento de sistemas e engenharia, o PMBOK® evoluiu ao longo das décadas para se tornar um padrão amplamente adotado em múltiplos setores da economia, incluindo tecnologia da informação, engenharia civil, indústria, saúde e serviços.

O PMBOK®, publicado e mantido pelo Project Management Institute (PMI®), consolida um conjunto de práticas, processos, ferramentas e técnicas reconhecidas como boas práticas no gerenciamento de projetos. Ele define uma estrutura conceitual que abrange áreas de conhecimento (como escopo, tempo, custo, qualidade, riscos, comunicações e integração) e grupos de processos (iniciação, planejamento, execução, monitoramento e controle, e encerramento), que podem ser adaptados conforme o contexto organizacional e a metodologia utilizada, incluindo abordagens preditivas, ágeis e híbridas.

Na engenharia de software, o PMBOK® atua como um arcabouço de governança que pode ser integrado a metodologias ágeis, fornecendo estrutura, rastreabilidade e alinhamento estratégico ao ciclo de vida do desenvolvimento. Essa integração permite equilibrar flexibilidade e controle, promovendo um ambiente em que a inovação tecnológica e a disciplina gerencial coexistam de forma harmônica.

A **6ª edição do PMBOK®**, publicada em 2017, consolidou uma visão estruturada baseada em 5 grupos de processos e 10 áreas de conhecimento, oferecendo um modelo sistemático para planejar e controlar projetos de maneira sequencial e preditiva. Essa abordagem foi especialmente útil em contextos onde o escopo e os requisitos eram bem definidos, como em projetos de infraestrutura ou sistemas de larga escala (PMBOK®, 2017).

Contudo, com o avanço das metodologias ágeis, o PMI® reconheceu a necessidade de um modelo mais flexível, adaptativo e centrado em valor. Assim, a **7ª edição do PMBOK® (2021)** introduziu uma mudança significativa: em vez de focar em processos fixos, passou a estruturar-se em **12 princípios de gerenciamento de projetos** e **8 domínios de desempenho**, enfatizando resultados, colaboração, aprendizado contínuo e entrega de valor ao cliente.

Importância do gerenciamento de projetos

O gerenciamento de projetos oferece uma abordagem estruturada, sistemática e orientada a resultados para o planejamento, execução, monitoramento e controle de projetos, garantindo eficiência e previsibilidade em todas as etapas do ciclo de vida.

- **Planejamento e organização:** fornece um framework sistemático para planejar, executar e controlar projetos de forma eficiente, estabelecendo objetivos claros, escopo definido, cronogramas realistas e alocação otimizada de recursos materiais, humanos, tecnológicos e financeiros, prevenindo retrabalhos, desperdícios e atrasos.
- **Gestão de riscos:** possibilita a identificação precoce e gestão proativa de riscos, análise e mitigação de riscos, reduzindo a probabilidade e o impacto de eventos indesejados sobre o desempenho do projeto.
- **Comunicação e alinhamento:** promove uma comunicação eficaz e colaborativa entre as equipes de desenvolvimento, stakeholders e clientes, assegurando o alinhamento das expectativas e o comprometimento com os objetivos do projeto.

- **Controle e melhoria contínua:** ao manter o projeto dentro do escopo, do prazo e do orçamento planejado, o gerenciamento de projetos aumenta a taxa de sucesso e a satisfação do cliente, além de gerar dados e aprendizados que alimentam um ciclo contínuo de aperfeiçoamento organizacional e tecnológico.

Em síntese, o gerenciamento de projetos atua como um instrumento estratégico de governança e qualidade, essencial para conectar a visão técnica da engenharia de software com os objetivos de negócio e inovação.

De acordo com a **6ª edição do Guia PMBOK® (2017)**, os projetos são compostos por diversos componentes-chave que se inter-relacionam de forma dinâmica ao longo do ciclo de vida do gerenciamento. Esses componentes, como escopo, tempo, custo, qualidade, recursos, comunicações, riscos e integração, formam um sistema interdependente que precisa ser continuamente monitorado e ajustado para garantir o sucesso do projeto.

Na engenharia de software, esses elementos assumem uma dimensão tecnológica essencial: o escopo se traduz em RF e RNF, o tempo é gerido por métodos iterativos e incrementais (como sprints) e a qualidade é sustentada por métricas automatizadas, CI/CD e testes baseados em ferramentas de monitoramento e versionamento.

Assim, o gerenciamento de projetos, conforme o PMBOK®, não se limita à gestão administrativa, mas constitui um arcabouço tecnológico e metodológico que integra planejamento estratégico, automação de processos, governança de dados e metodologias ágeis, promovendo a eficiência e previsibilidade na entrega de soluções de software.

O quadro a seguir mostra esses componentes e suas inter-relações dentro do ciclo de vida do projeto, evidenciando como a gestão integrada de processos e tecnologia sustenta o desempenho organizacional.

Quadro 14 – Componentes-chave do Guia PMBOK®

Ciclo de vida do projeto
Fase do projeto
Revisão de fase
Grupo de processos de gerenciamento de projetos
Processo de gerenciamento de projetos
Área de conhecimento em gerenciamento de projetos

Fonte: PMBOK® (2017).

Como mostra o gráfico ilustrado na figura 27, o ciclo de vida do projeto, conforme descrito no Guia PMBOK®, é composto por cinco grupos de processos: grupo de processos de iniciação, de processos de planejamento, de processos de execução, de processos de monitoramento e controle, e de processos de encerramento.

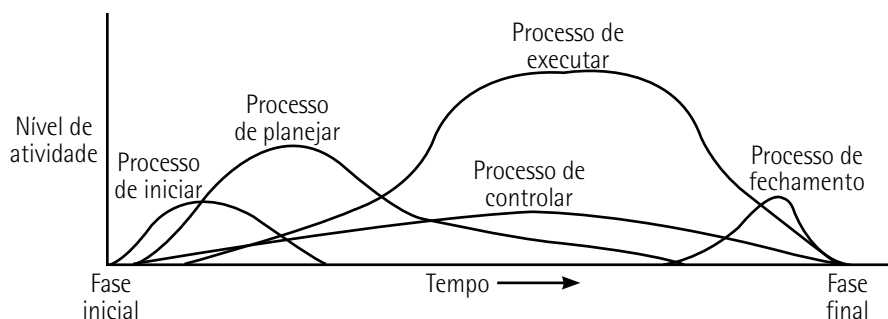


Figura 27 – Interações entre os grupos de processos dentro do projeto

Fonte: PMBOK® (2017).

Cada uma dessas etapas representa um conjunto estruturado de atividades que orienta o gerente de projetos e a equipe no planejamento, acompanhamento e controle das entregas.

- **Grupo de processos de iniciação:** tem como objetivo definir um novo projeto ou uma nova fase de um projeto existente. Nessa etapa, são realizadas análises de viabilidade, identificação das partes interessadas e levantamento dos requisitos iniciais de escopo. O resultado é a autorização formal para o início do projeto.
- **Grupo de processos de planejamento:** envolve o desenvolvimento de um plano detalhado que estabelece os objetivos, o escopo, as atividades, os recursos e o cronograma do projeto. Também compreendem a identificação de riscos, a definição de estratégias de mitigação, o plano de comunicação e o alinhamento com as partes interessadas. Essa etapa resulta em um plano abrangente que orienta a execução e o controle do projeto, garantindo coerência e direcionamento para o sucesso.
- **Grupo de processos de execução:** corresponde à implementação do que foi planejado, com a realização das atividades necessárias para atingir os objetivos do projeto. É nessa fase que os planos se transformam em entregas concretas, promovendo a coordenação de pessoas, recursos e tarefas para alcançar os resultados esperados.
- **Grupo de processos de monitoramento e controle:** constituem uma fase essencial do gerenciamento de projetos, voltada ao acompanhamento do desempenho, controle das atividades e gestão de mudanças. O objetivo é garantir que o projeto permaneça dentro do escopo, do prazo e do orçamento definidos. Essa etapa permite a identificação de desvios e a aplicação de ações corretivas ou preventivas, assegurando o alinhamento contínuo aos objetivos estabelecidos.
- **Grupo de processos de encerramento:** marcam a conclusão formal do projeto ou de uma fase. Nessa etapa, as entregas são verificadas e aceitas pelos stakeholders, contratos são encerrados e recursos são liberados. Além disso, são documentadas as lições aprendidas e elaborados relatórios finais. O encerramento é fundamental para consolidar o sucesso do projeto e registrar o conhecimento adquirido para futuras iniciativas.

Além dos grupos de processos, o PMBOK® também organiza o gerenciamento de projetos em áreas de conhecimento.



Lembrete

Como orientação ao projeto de sistemas de software, a integração dos processos do PMBOK® com os processos do ciclo de vida do software da NBR ISO/IEC 12207 permite compreender de forma mais ampla como estruturar a organização do projeto, definir papéis, responsabilidades e relacionamentos, e aplicar boas práticas de engenharia de software dentro de um contexto de governança tecnológica e metodológica.

Cada área de conhecimento representa um domínio especializado dentro da gestão de projetos, estruturado a partir de um conjunto de processos inter-relacionados, descritos em termos de suas boas práticas, entradas, saídas, ferramentas e técnicas associadas.

Essas áreas funcionam como campos de competência técnica e gerencial, permitindo ao gestor compreender e aplicar, de forma integrada, os fundamentos do planejamento, execução, monitoramento e controle necessários para o sucesso do projeto. Dessa forma, conforme o Guia de Projeto PMBOK®, destacamos os cinco grupos de processos e as dez áreas de conhecimento, como mostra o quadro a seguir, que apresenta também as atividades a serem desenvolvidas em cada etapa.

Quadro 15 – Grupo de processos de gerenciamento de projetos e mapeamento das áreas de conhecimento

Áreas de conhecimento	Grupos de processos de gerenciamento de projetos				
	Grupo de processos de iniciação	Grupo de processos de planejamento	Grupo de processos de execução	Grupo de processos de monitoramento e controle	Grupo de processos de encerramento
4. Gerenciamento da integração do projeto	4.1 Desenvolver o termo de abertura do projeto	4.2 Desenvolver o Plano de Gerenciamento do Projeto	4.3 Orientar e gerenciar o trabalho do projeto 4.4 Gerenciar o conhecimento do projeto	4.5 Monitorar e controlar o trabalho do projeto 4.6 Realizar o controle integrado de mudanças	4.7 Encerrar o projeto ou fase
5. Gerenciamento do escopo do projeto		5.1 Planejar o gerenciamento do escopo 5.2 Coletar os requisitos 5.3 Definir o escopo 5.4 Criar a EAP		5.5 Validar o escopo 5.6 Controlar o escopo	

Áreas de conhecimento	Grupos de processos de gerenciamento de projetos				
	Grupo de processos de iniciação	Grupo de processos de planejamento	Grupo de processos de execução	Grupo de processos de monitoramento e controle	Grupo de processos de encerramento
6. Gerenciamento do cronograma do projeto		6.1 Planejar o gerenciamento do cronograma 6.2 Definir as atividades 6.3 Sequenciar as atividades 6.4 Estimar as durações das atividades 6.5 Desenvolver o cronograma		6.6 Controlar o cronograma	
7. Gerenciamento dos custos do projeto		7.1 Planejar o gerenciamento dos custos 7.2 Estimar os custos 7.3 Determinar o orçamento		7.4 Controlar os custos	
8. Gerenciamento da qualidade do projeto		8.1 Planejar o gerenciamento da qualidade	8.2 Gerenciar a qualidade	8.3 Controlar a qualidade	
9. Gerenciamento dos recursos do projeto		9.1 Planejar o gerenciamento dos recursos 9.2 Estimar os recursos das atividades	9.3 Adquirir recursos 9.4 Desenvolver a equipe 9.5 Gerenciar a equipe	9.6 Controlar os recursos	
10. Gerenciamento das comunicações do projeto		10.1 Planejar o gerenciamento das comunicações	10.2 Gerenciar as comunicações	10.3 Monitorar as comunicações	
11. Gerenciamento dos riscos do projeto		11.1 Planejar o gerenciamento dos riscos 11.2 Identificar os riscos 11.3 Realizar a análise qualitativa dos riscos 11.4 Realizar a análise quantitativa dos riscos 11.5 Planejar as respostas aos riscos	11.6 Implementar respostas aos riscos	11.7 Monitorar os riscos	
12. Gerenciamento das aquisições do projeto		12.1 Planejar o gerenciamento das aquisições	12.2 Conduzir as aquisições	12.3 Controlar as aquisições	
13. Gerenciamento das partes interessadas do projeto	13.1 Identificar as partes interessadas	13.2 Planejar o engajamento das partes interessadas	13.3 Gerenciar o engajamento das partes interessadas	13.4 Monitorar o engajamento das partes interessadas	

Adaptado de: PMBOK® (2017).

Cada área de conhecimento (ou gerência) é associada a um ou mais processos do grupo de processos de influência no projeto. Todas essas relações são descritas detalhadamente no Guia PMBOK®.

Exemplo de aplicação

Estudo de caso: iniciação do projeto

Identifique no quadro anterior a coluna "Grupo de processos de iniciação" e a seguir identifique a área de conhecimento "4. Gerenciamento da integração do projeto". A célula referente ao processo ou atividade a ser realizada corresponde ao registro na célula "4.1 Desenvolver o termo de abertura do projeto". Consulte o guia de projeto PMBOK® (2017) e veja a seção de mesma numeração e nome:

4.1 Desenvolver o termo de abertura do projeto: o processo de desenvolver um documento que formalmente autoriza a existência de um projeto e fornece ao gerente do projeto a autoridade necessária para aplicar recursos organizacionais às atividades do projeto (PMBOK®, 2017, p. 70).

Quadro 16 – Processo do guia PMBOK® para desenvolver o termo de abertura do projeto

4.1 Desenvolver o Termo de Abertura do Projeto
1) Entradas
1 Documentos de negócios
2 Acordos
3 Fatores ambientais da empresa
4 Ativos de processos organizacionais
2) Ferramentas e técnicas
1 Opinião especializada
2 Coleta de dados
3 Habilidades interpessoais e de equipe
4 Reuniões
3) Saídas
1 Termo de abertura do projeto
2 Registro de premissas

Adaptado de: PMBOK® (2017).

Observe que todo o processo no Guia PMBOK® possui essa estrutura: entradas, ferramentas e técnicas, e saídas. Cada uma dessas atividades está descrita com detalhes de como desenvolvê-la e efetivá-la.

5.2.1 Abordagens adaptativas e metodologias ágeis no gerenciamento de projetos

A integração do PMBOK® com abordagens ágeis levou à **7ª edição do Guia PMBOK® (2021)**, que estabelece um comparativo entre o planejamento preditivo e o planejamento adaptativo. Como dito anteriormente, a 7ª edição passou a estruturar-se em 12 princípios de gerenciamento de projetos e 8 domínios de desempenho.

O Guia PMBOK® (2021, p. 21) afirma:

[...] Os **princípios do gerenciamento de projetos** não são de natureza prescritiva. Pretendem apenas orientar o comportamento das pessoas envolvidas com projetos. Eles têm uma base ampla, então há muitas maneiras de as pessoas e as organizações manterem o alinhamento com os princípios.

As **abordagens adaptativas**, conforme descritas no PMBOK® 7ª edição, são especialmente eficazes em contextos em que os requisitos do projeto apresentam alto grau de incerteza, complexidade e volatilidade. Nesses casos, mudanças ao longo do ciclo de vida são esperadas e até desejadas, pois permitem melhor adequação às necessidades reais do cliente e às condições do ambiente.

Na abordagem adaptativa utilizam-se ciclos iterativos e incrementais, promovendo entregas parciais e contínuas de valor. A cada iteração, o produto é aprimorado com base em aprendizados obtidos, revisões e validações com as partes interessadas, fortalecendo a colaboração, a transparência e a flexibilidade.

As metodologias ágeis representam uma aplicação prática das abordagens adaptativas. Embora a agilidade seja uma mentalidade mais ampla do que um simples framework de desenvolvimento, suas práticas se alinham perfeitamente aos princípios adaptativos do PMBOK®. Frameworks ágeis como Scrum, XP, Kanban, FDD e outros promovem **ciclos curtos de trabalho** com iterações que ocorrem durante todo o ciclo de desenvolvimento, normalmente de uma a duas semanas, em que são realizadas revisões constantes, planejamento colaborativo e feedback contínuo.

Em cada iteração, a equipe do projeto seleciona o conjunto de atividades que pode ser entregue com base em um backlog priorizado, que estima o esforço necessário e trabalha de forma integrada para atingir o escopo planejado. Ao término do ciclo, realiza-se uma revisão dos resultados, permitindo ajustes imediatos e garantindo a evolução contínua do produto.

A integração entre o PMBOK® 7ª edição e as metodologias ágeis representa uma evolução natural do gerenciamento de projetos, combinando estrutura e adaptabilidade. Essa sinergia proporciona maior capacidade de resposta a mudanças, foco no valor entregue e engajamento efetivo das equipes, conduzindo a projetos mais eficientes, colaborativos e orientados a resultados reais.

Os 12 princípios de gerenciamento de projetos do PMBOK® 7ª edição são (PMBOK®, 2021):

- 1) **Seja um profissional diligente, respeitoso e atencioso:** o gerente de projetos deve agir com integridade, responsabilidade e ética, valorizando as pessoas e os recursos. O comportamento profissional influencia diretamente a credibilidade e o sucesso do projeto.
- 2) **Crie um ambiente colaborativo para a equipe:** promover uma cultura de confiança, respeito e engajamento é essencial. A equipe deve sentir-se segura para compartilhar ideias, levantar riscos e propor melhorias.
- 3) **Envolva-se de fato com as partes interessadas:** as partes interessadas (stakeholders) devem ser continuamente identificadas, engajadas e ouvidas. A comunicação eficaz é vital para alinhar expectativas e sustentar o apoio ao projeto.
- 4) **Enfoque no valor:** todo projeto deve gerar valor mensurável para as partes interessadas. O gerente precisa compreender o que é valor para cada stakeholder e orientar decisões que maximizem esse retorno.
- 5) **Reconheça, avalie e reaja às interações do sistema:** os projetos operam dentro de sistemas dinâmicos (organizacionais, sociais e tecnológicos). O profissional deve compreender essas inter-relações e adaptar o projeto de forma sistêmica, e não isolada.
- 6) **Demonstre comportamentos de liderança:** o ambiente de projetos envolve riscos, variabilidade e interdependências. O gerente deve estar preparado para atuar com flexibilidade e adotar práticas adaptativas conforme o contexto.
- 7) **Faça a adaptação de acordo com o contexto:** cada projeto é único. As práticas e métodos devem ser personalizadas de acordo com o tamanho, complexidade, riscos e ambiente do projeto, evitando a aplicação rígida de frameworks.
- 8) **Inclua qualidade nos processos e nas entregas:** a qualidade deve ser planejada e incorporada desde o início, não apenas verificada no final. O foco está em prevenir falhas, promover melhorias contínuas e garantir a satisfação do cliente.
- 9) **Complexidade:** os projetos operam em ambientes complexos e dinâmicos, exigindo do gerente a capacidade de compreender, lidar e aprender com essa complexidade. É fundamental promover a reflexão contínua, o compartilhamento de conhecimento e a melhoria constante dos processos, estimulando o aprendizado tanto individual quanto organizacional.
- 10) **Otimize as respostas aos riscos:** gerenciar riscos é um processo proativo e contínuo. Envolve identificar oportunidades e ameaças, e implementar respostas que aumentem a probabilidade de sucesso do projeto.
- 11) **Adote a capacidade de adaptação e resiliência:** projetos enfrentam mudanças e imprevistos e devem ser flexíveis, com aprendizado rápido e ajustado a novas condições, sendo essencial para que se possa manter a continuidade e alcançar os objetivos.

- 12) **Aceite a mudança para alcançar o futuro estado previsto:** o sucesso de um projeto não se resume à entrega pontual. Ele deve gerar benefícios duradouros, respeitando aspectos sociais, ambientais e econômicos, e contribuindo para o desenvolvimento sustentável.



Observação

Os princípios de gerenciamento de projetos constituem a base conceitual para a condução eficaz de iniciativas organizacionais, orientando decisões, comportamentos e práticas ao longo do ciclo de vida do projeto. Sua aplicação favorece o alinhamento entre objetivos estratégicos e execução operacional, a gestão adequada de riscos e recursos, bem como a adaptação a mudanças de contexto. Ao fornecer diretrizes que transcendem métodos e ferramentas específicas, esses princípios contribuem para a previsibilidade, a governança e a geração consistente de valor para as partes interessadas.

Os 8 domínios de desempenho do PMBOK® 7ª edição são (PMBOK®, 2021):

- 1) **Partes interessadas (stakeholders):** foca na identificação, análise, engajamento e comunicação com as partes interessadas ao longo de todo o ciclo de vida do projeto. O objetivo é alcançar alinhamento e comprometimento, reduzindo resistências e ampliando o suporte ao projeto.
- 2) **Equipe (team):** trata da formação, desenvolvimento e liderança da equipe do projeto. Envolve promover colaboração, confiança, diversidade e empoderamento, criando um ambiente de alto desempenho e foco nos resultados.
- 3) **Abordagem de desenvolvimento e do ciclo de vida (development approach and life cycle):** refere-se à escolha e aplicação da abordagem mais adequada (preditiva, adaptativa ou híbrida) e à definição do ciclo de vida do projeto (fases, marcos e entregas). Busca equilibrar flexibilidade e controle, conforme a natureza do projeto e o grau de incerteza.
- 4) **Planejamento (planning):** envolve o planejamento contínuo e adaptável do projeto, considerando objetivos, escopo, cronograma, recursos, riscos e comunicações. O foco está em ajustar o plano conforme novas informações surgem, mantendo o projeto alinhado às metas estratégicas e ao valor esperado.
- 5) **Trabalho do projeto (project work):** abrange a execução, coordenação e monitoramento das atividades que produzem as entregas. Inclui o gerenciamento da qualidade, recursos, aquisições e comunicações, assegurando que o trabalho seja realizado de forma eficaz e eficiente.
- 6) **Entrega (delivery):** foca na entrega de resultados e benefícios que atendam às expectativas das partes interessadas e gerem valor real para a organização. Inclui validar entregas, gerenciar aceites e garantir que os objetivos estratégicos sejam efetivamente alcançados.

7) Medição (measurement): trata da avaliação do desempenho do projeto por meio de métricas, indicadores e informações quantitativas e qualitativas. O objetivo é monitorar o progresso, apoiar decisões e promover melhorias contínuas, assegurando a transparência dos resultados.

8) Incerteza (uncertainty): envolve a identificação e o gerenciamento proativo de riscos, oportunidades e variabilidades. Promove uma mentalidade adaptativa e resiliente, permitindo que a equipe responda com agilidade às mudanças e minimize impactos negativos.

Esses oito domínios se inter-relacionam e devem ser aplicados de forma integrada e contínua, não linear. Eles servem como um sistema completo de gestão, no qual o desempenho em um domínio influencia diretamente os demais, refletindo o caráter dinâmico e adaptável do gerenciamento de projetos moderno.

5.3 Guia de referência para engenharia de software SWEBOK®

O **Software Engineering Body of Knowledge (SWEBOK®)**, traduzido como Corpo de Conhecimento em Engenharia de Software, é um guia internacional que consolida o conhecimento essencial da engenharia de software, padronizando termos, práticas e princípios reconhecidos pela comunidade profissional. Ele foi desenvolvido sob a coordenação do IEEE Computer Society, com apoio do ISO/IEC, e tem como objetivo principal definir o escopo e o corpo de conhecimento que todo engenheiro de software deve dominar.

O Guia SWEBOK® tem por finalidade servir de referência global para a formação, certificação e prática profissional em engenharia de software. Ele não dita como o trabalho deve ser feito, mas define o que deve ser conhecido e compreendido, funcionando como uma base de referência internacional para o desenvolvimento, ensino e gestão de software.

Seu propósito é:

- padronizar a terminologia e os conceitos fundamentais da área;
- definir competências essenciais para engenheiros de software;
- orientar currículos acadêmicos, programas de certificação e avaliações de competência;
- promover a uniformidade e o reconhecimento da engenharia de software como uma disciplina de engenharia legítima.

O Guia SWEBOK® foi estabelecido com os seguintes cinco objetivos:

- promover uma visão consistente da engenharia de software em âmbito mundial;
- especificar o escopo e esclarecer a posição da engenharia de software em relação a outras disciplinas, como ciência da computação, gerenciamento de projetos, engenharia da computação e matemática;

- caracterizar o conteúdo da disciplina de engenharia de software;
- fornecer acesso temático ao corpo de conhecimento em engenharia de software;
- prover uma base para o desenvolvimento curricular e para a formação individual.

Desses cinco objetivos, o segundo, que é especificar o escopo da engenharia de software, constitui a base da organização fundamental do Guia. De acordo com Washizaki (2025), o conteúdo do Guia consiste em 18 áreas de conhecimento (knowledge areas – KAs) que são seguidas por vários apêndices.

Uma área de conhecimento é identificada na engenharia de software que descreve em termos de conhecimentos já aceitos, incluindo processos, componentes, práticas, entradas, saídas, ferramentas e técnicas. Cada área de conhecimento é tratada como um capítulo no Guia, sendo divididas em áreas de conhecimento da engenharia de software e conhecimentos fundamentais sobre as áreas de conhecimento. Essas KAs representam domínios de atuação profissional.

As 18 áreas de conhecimento (KAs) do SWEBOK® v4.0 são:

- 1) Requisitos de software (software requirements):** trata da identificação, análise, especificação, validação e gestão dos requisitos de software. Define o que o sistema deve fazer e as restrições sob as quais deve operar.
- 2) Arquitetura de software (software architecture):** aborda o projeto e a estrutura de alto nível de um sistema, incluindo componentes, interações e decisões arquiteturais que impactam desempenho, segurança e manutenção.
- 3) Projeto de software (software design):** foco na transformação dos requisitos em uma solução detalhada, especificando componentes, algoritmos, interfaces e dados. É a base para a implementação do sistema.
- 4) Construção de software (software construction):** compreende as atividades de codificação, verificação unitária e integração, aplicando boas práticas de programação, padrões de código e ferramentas de desenvolvimento.
- 5) Teste de software (software testing):** engloba as técnicas e processos usados para avaliar a qualidade e verificar se o software atende aos requisitos. Inclui testes unitários, de integração, de sistema e de aceitação.
- 6) Operações de engenharia de software (software engineering operations):** trata das atividades de implantação, operação e suporte do software em ambiente de produção, incluindo monitoramento, automação e continuidade de serviços.
- 7) Manutenção de software (software maintenance):** abrange as atividades pós-entrega, como correção de defeitos, melhorias e adaptações do software para novos ambientes ou necessidades do usuário.

- 8) **Gerenciamento de configuração de software (software configuration management):** responsável pelo controle de versões, rastreabilidade e integridade dos artefatos do projeto, garantindo que alterações sejam gerenciadas de forma sistemática.
- 9) **Gerenciamento de engenharia de software (software engineering management):** envolve o planejamento, acompanhamento e controle de projetos de software, incluindo escopo, cronograma, custos, riscos e qualidade, para garantir o sucesso da entrega.
- 10) **Processo de engenharia de software (software engineering process):** define e melhora os processos utilizados para desenvolver e manter software, promovendo padronização, eficiência e melhoria contínua organizacional.
- 11) **Modelos e métodos de engenharia de software (software engineering models and methods):** inclui os modelos de ciclo de vida (como cascata, incremental e ágil) e os métodos formais ou empíricos aplicados ao desenvolvimento e análise de software.
- 12) **Qualidade de software (software quality):** foca na garantia e avaliação da qualidade, cobrindo normas, métricas, revisões e auditorias que asseguram que o produto atenda às expectativas e requisitos definidos.
- 13) **Segurança de software (software security):** trata da proteção do software contra vulnerabilidades, ataques e falhas, incorporando princípios de segurança desde o projeto até a operação.
- 14) **Prática profissional em engenharia de software (software engineering professional practice):** aborda a ética, comunicação, trabalho em equipe, responsabilidade social e competências profissionais necessárias para a prática responsável da engenharia de software.
- 15) **Economia da engenharia de software (software engineering economics):** analisa o impacto econômico das decisões de engenharia, considerando custos, benefícios, riscos e trade-offs (relação de compromisso) ao longo do ciclo de vida do software.
- 16) **Fundamentos de computação (computing foundations):** reúnem os conceitos básicos de ciência da computação como estruturas de dados, algoritmos, arquitetura de computadores e sistemas operacionais, que sustentam a engenharia de software.
- 17) **Fundamentos matemáticos (mathematical foundations):** inclui os conceitos matemáticos e lógicos aplicados à modelagem, análise e verificação de software, como álgebra, estatística, lógica formal e teoria dos grafos.
- 18) **Fundamentos de engenharia (engineering foundations):** apresenta os princípios gerais da engenharia, como análise de sistemas, modelagem, controle de qualidade, segurança e ética, aplicados ao contexto do desenvolvimento de software.

5.3.1 Panorama dos padrões de engenharia de software

A figura a seguir apresenta um panorama dos padrões de engenharia de software que mais se destacam, principalmente da perspectiva de como outros padrões se relacionam com o principal padrão de processo de ciclo de vida de engenharia de software, o ISO/IEC/IEEE 12207 (Processos de engenharia de software), servindo como ponto central para a arquitetura de padrões de engenharia de software (veja na unidade II o tópico 3.1 Processos do ciclo de vida software: NBR ISO/IEC 12207). A relação com o SWEBOK® ocorre no sentido de que ambos apresentam informações relacionadas a muitos dos mesmos processos do ciclo de vida de software.

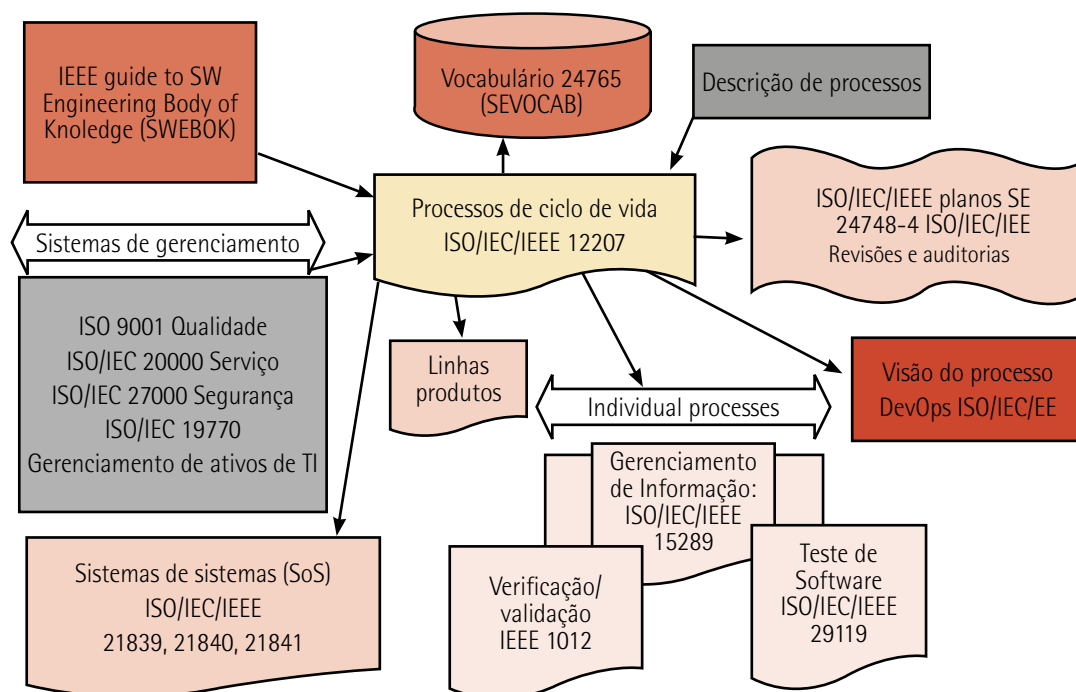


Figura 28 – Panorama dos padrões de engenharia de software

Adaptada de: Washizaki (2025).

Na parte superior da figura 28, estão os padrões fundamentais, como o vocabulário especializado para engenharia de sistemas e software (Sevocab, ISO/IEC/IEEE 24765) e uma especificação sobre como descrever processos (ISO/IEC/IEEE 24774).

Existem padrões sobre como **planejar e gerenciar** a engenharia de software (ISO/IEC/IEEE 24748-5) e como conduzir **revisões e auditorias** rigorosas, apropriadas para softwares críticos como sistemas aeroespaciais e de defesa (ISO/IEC/IEEE 24748-8).

Além disso, existem muitos outros padrões especializados cobrindo processos individuais e abordagens modernas aos processos, como o ISO/IEC/IEEE 32675 para DevOps, bem como o IEEE 1012 para **verificação/validação** e o ISO/IEC/IEEE 29119 para **teste de software** (em múltiplas partes).

As séries mais especializadas se concentram em processos e ferramentas para **engenharia de linha de produto** e para **sistemas de sistemas (SoS)**. Os padrões de sistema de sistemas ISO/IEC/IEEE 21839, 21840 e 21841 explicam como usar processos de engenharia de sistemas quando o sistema de interesse (SOI) é uma parte constituinte de um sistema de sistemas.

Os padrões de processo de ciclo de vida são projetados para serem compatíveis com outros padrões bem conhecidos de **sistemas de gestão**, de acordo com a ISO. Os sistemas de gerenciamento possuem uma estrutura e um framework de requisitos consistentes, e cada um aborda um aspecto específico de gestão, entrega de produtos e serviços de engenharia. Os sistemas de gerenciamento são apresentados em vários guias para diferentes aspectos de seus sistemas. Os principais sistemas de gerenciamento relacionados à engenharia de software incluem:

- ISO 9001 para gestão da qualidade (veja na unidade II o tópico 3.4 Sistemas da qualidade: ISO 9001 e ISO 90003);
- ISO/IEC 20000 para gestão de serviços;
- a série ISO/IEC 27000 para gestão de segurança da informação;
- a série ISO/IEC 19770 para gestão de ativos de TI (como hardware e software).

5.4 Arquiteturas de sistemas: uma abordagem integrada entre PMBOK® e SWEBOK®

No planejamento estratégico de projetos (abordado pelo PMBOK®), a arquitetura de sistemas atua como uma ponte para a execução técnica da engenharia de software (abordada pelo SWEBOK®).

O PMBOK® é o guia de boas práticas para o gerenciamento de projetos. Na arquitetura de sistemas, ele lida desde a concepção, o controle do ciclo de desenvolvimento, a entrega do produto software (deliverable) e o fator de risco/qualidade, conforme vimos no tópico 5.2 Guia do conhecimento em gerenciamento de projetos PMBOK®.

Principais áreas de conhecimento aplicadas:

- **Gerenciamento de integração do projeto:** a arquitetura fornece o plano mestre técnico que integra o trabalho de diferentes equipes (desenvolvimento backend, frontend, banco de dados etc.), garantindo que todos os componentes funcionem como um sistema coeso.
- **Gerenciamento do escopo do projeto:** a arquitetura é um dos principais produtos de entrega do projeto de software. O design arquitetural precisa ser claramente definido no escopo para que o trabalho possa ser planejado e executado.
- **Gerenciamento da qualidade do projeto:** as decisões arquiteturais afetam diretamente a qualidade do produto final. Atributos como escalabilidade, desempenho e segurança (que são definidos na arquitetura) são verificados e validados por meio de processos de qualidade do PMBOK®.

- **Gerenciamento dos riscos do projeto:** a inadequação ou a não documentação da arquitetura é um risco significativo para o projeto. O PMBOK® exige que o gerente de projetos identifique, analise e planeje respostas para riscos arquiteturais (por exemplo: risco da arquitetura em termos de CPU e rede não suportarem o volume de usuários esperados).

Assim, compreendemos que o PMBOK® trata a arquitetura de sistemas como o **artefato-chave** que impacta no escopo, tempo, custo e qualidade do projeto, garantindo que o planejamento e a execução gerencial apoiem a correta implementação da arquitetura definida.

O SWEBOK®, por sua vez, é o guia de conhecimento para a engenharia de software. Na arquitetura de sistemas, é definida em seu nível mais técnico e conceitual:

- **Área de conhecimento "projeto de software" (software design):** a arquitetura é o componente central dessa área. O SWEBOK® define arquitetura de software como a atividade do ciclo de vida em que os requisitos são analisados para produzir uma descrição da estrutura interna do software (componentes, interfaces e suas relações).
- **Definição de estrutura:** o SWEBOK® aborda estilos arquiteturais (como cliente-servidor, camadas, microserviços), padrões e a tomada de decisões sobre a estrutura que garantirá os atributos de qualidade (como desempenho, segurança e manutenibilidade).

O SWEBOK® define **o que** deve ser feito para conceber uma arquitetura de software de qualidade e **como** essa arquitetura deve ser documentada.

A integração entre os referenciais PMBOK® e SWEBOK® oferece uma base sólida para compreender o papel estratégico da arquitetura de sistemas no desenvolvimento de software. O PMBOK® orienta o gerenciamento de projetos com foco em escopo, tempo, qualidade e riscos, e o SWEBOK® sistematiza as práticas da engenharia de software, incluindo a definição e avaliação de arquiteturas.

A arquitetura de sistemas surge como um elo entre gestão e engenharia, sendo responsável por estruturar os componentes, definir padrões de interação e garantir atributos de qualidade como desempenho, escalabilidade e manutenibilidade. O entendimento dos elementos constituintes e dos tipos de arquitetura é essencial para alinhar decisões técnicas aos objetivos do projeto, promovendo soluções robustas e sustentáveis.

5.4.1 Arquitetura de sistemas: elementos, tipos e impactos na qualidade

Arquiteturas de sistemas são estruturas organizacionais que definem como os componentes de um sistema interagem, se comunicam e funcionam em conjunto para atender aos requisitos de negócio e técnicos. Ela descreve a estrutura, comportamento e relações entre os elementos de um sistema, como módulos, serviços, interfaces e protocolos. A arquitetura do sistema funciona como um plano mestre que orienta o desenvolvimento, integração e evolução do sistema ao longo do tempo. São fundamentais para garantir eficiência, escalabilidade e qualidade no desenvolvimento de software.

Com uma visão de alto nível, ela representa um mapa estratégico que vai muito além do código-fonte. É a abstração do sistema, permitindo que a equipe entenda como as grandes peças se encaixam e interagem. Com base na arquitetura do sistema, o arquiteto toma **decisões críticas** sobre tecnologias, padrões e a forma de organização que irão guiar todo o desenvolvimento e a evolução do produto.

A arquitetura de sistemas é o projeto estrutural que sustenta o ambiente computacional, englobando a definição dos componentes principais, suas propriedades visíveis externamente e os relacionamentos entre eles. Essa estrutura não apenas organiza tecnicamente o sistema, mas também exerce influência direta sobre os atributos de qualidade do produto de software, como desempenho, escalabilidade, segurança e manutenibilidade.

A clareza na definição arquitetural permite que decisões técnicas sejam tomadas com base em critérios de qualidade, alinhando os objetivos do projeto às expectativas dos usuários e às boas práticas da engenharia de software. Compreender os principais elementos da arquitetura de sistemas torna-se essencial para garantir soluções robustas, eficientes e sustentáveis.

Os principais elementos da arquitetura de sistemas são:

- **Camadas:** separação lógica entre apresentação, lógica de negócio e integração (ou dados), com foco no negócio. Veja a figura a seguir:

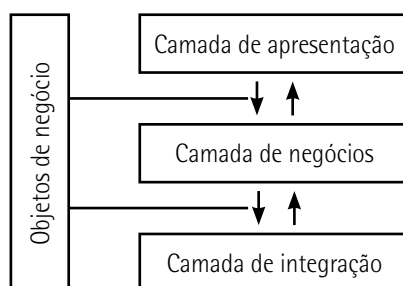


Figura 29 – Arquitetura em camadas

Fonte: Versolatto (2015).

- **Serviços:** funcionalidades específicas que o sistema de software oferece.
- **Interfaces:** pontos de interação entre componentes ou com o usuário.
- **Protocolos de comunicação:** regras, normalmente de rede de computadores para troca de dados entre os módulos do sistema de software.
- **Padrões de design:** soluções reutilizáveis para problemas recorrentes.

Já os tipos de arquitetura de sistemas são:

- **Monolítico:** todos os componentes integrados em um único bloco, servindo como base para a maioria dos sistemas no início da computação moderna, em que a simplicidade de implantação em uma única unidade era mais importante que a escalabilidade.

A figura a seguir apresenta um exemplo de arquitetura monolítica para atendimento de terminais como PDV, leitores de códigos de barras, controle de catracas, sinalizadores e outros. Observe que é no "computador servidor" que é executada a aplicação e o gerenciamento do banco de dados.

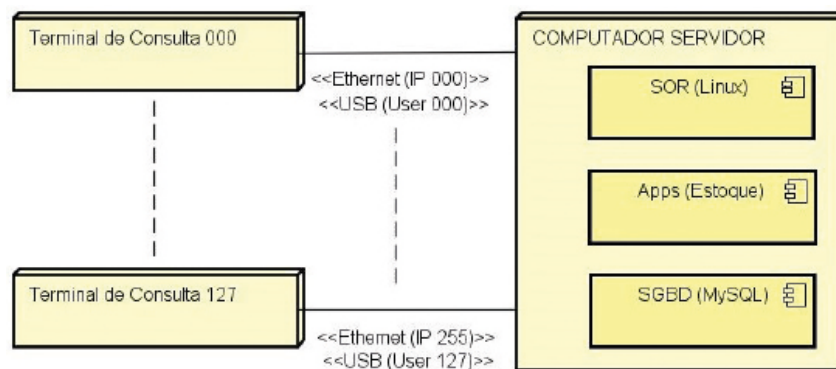


Figura 30 – Arquitetura monolítica para um sistema multiusuário e monotarefa para terminais de consulta

- **Cliente-servidor:** separação entre quem solicita e quem fornece os serviços, estabelecendo o primeiro grande padrão de distribuição básica e separando a apresentação (cliente) da lógica e dos dados (servidor).

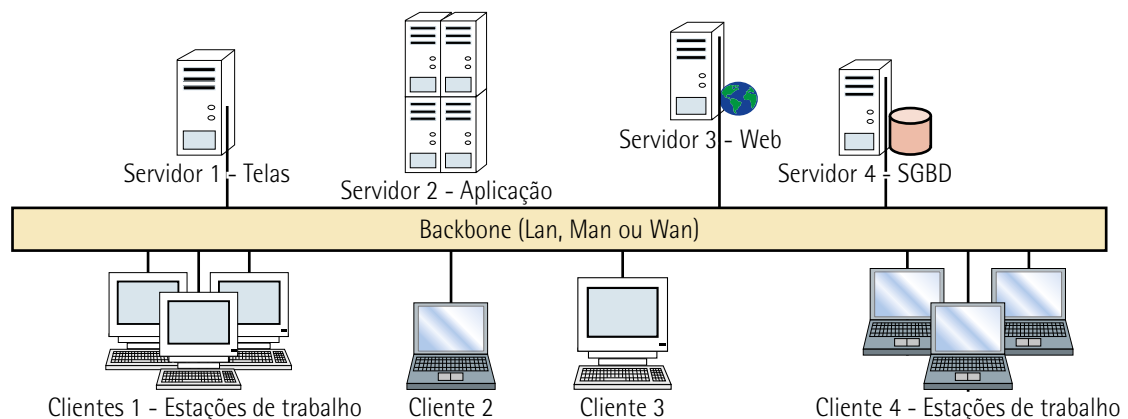


Figura 31 – Arquitetura cliente/servidor

- **Arquitetura em camadas:** organização hierárquica por responsabilidades com o foco em resolver problemas de manutenibilidade e organização dentro de um sistema de software, que pode ser também um monólito. Estabelecendo regras claras de comunicação e responsabilidades. Os principais módulos: apresentação → negócios → dados, apresentados na figura a seguir (figura 32), representam as principais classes a serem projetadas.

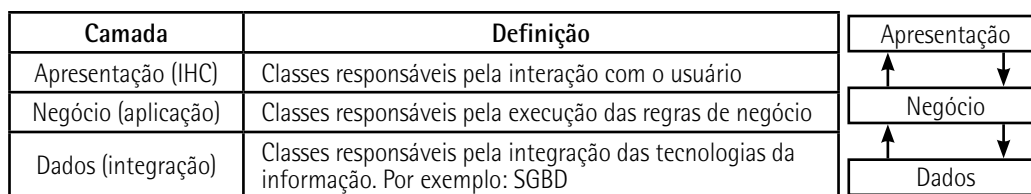


Figura 32 – Modelo de arquitetura em camadas

- **Arquitetura orientada a serviços (SOA):** são componentes de serviços independentes (componentização), introduzindo a ideia de grandes serviços reusáveis e permitindo o desacoplamento e a integração entre sistemas corporativos grandes e heterogêneos. O foco está na reutilização em nível empresarial.



Saiba mais

Leia o artigo a seguir sobre aplicação da arquitetura SOA:

FUGITA, H. S.; CARVALHO JÚNIOR, D. S. Método de análise e projetos em SOA. *DevMedia*, 2010. Disponível em: <https://tinyurl.com/4y8ftzdf>. Acesso em: 22 dez. 2025.

- **Microserviços:** divisão em pequenos serviços autônomos e escaláveis, sendo uma evolução direta do SOA com foco em serviços menores, independentes, autônomos (com seus próprios bancos de dados e módulos de implantação) e na comunicação leve (geralmente HTTP/REST ou envio de mensagens). O foco está na velocidade de disponibilidade e escalabilidade individual. A arquitetura de microserviços é, atualmente, a arquitetura projetada mais alinhada com os princípios de **sistemas distribuídos**.

A figura a seguir apresenta uma arquitetura de serviços distribuídos, que serve de apoio à internet, comum nas aplicações do e-commerce. Esse é um modelo conceitual de abstração de alto nível sem detalhes específicos de como vai funcionar.

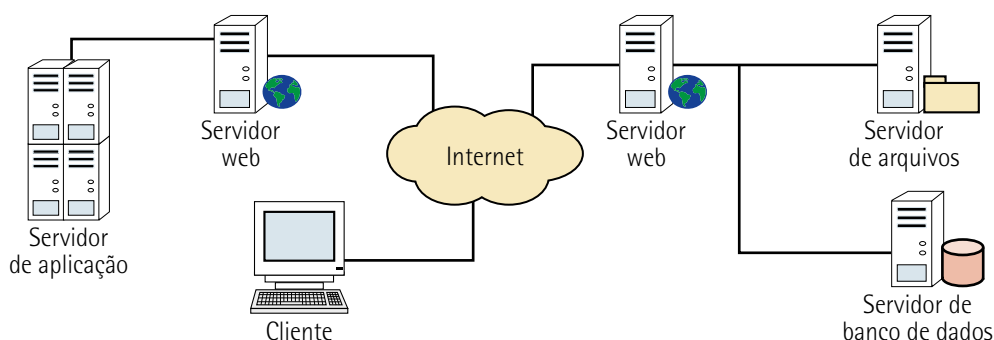


Figura 33 – Projeto conceitual de arquitetura de um sistema distribuído para internet

6 ECOSISTEMAS DIGITAIS E ENGENHARIA DE SOFTWARE CONECTADA

O desenvolvimento de software atual ocorre em um cenário cada vez mais interconectado, dinâmico e orientado a serviços. Os **ecossistemas digitais** representam esse novo ambiente de criação e evolução tecnológica, no qual múltiplas plataformas, organizações, usuários e tecnologias interagem de forma contínua. Nesses ecossistemas, o software não é mais um produto isolado, mas um componente vivo de uma rede maior, que integra aplicações, bancos de dados, APIs, dispositivos, serviços distribuídos em nuvem e outros.

A engenharia de software conectada surge, assim, como uma abordagem que combina visão de negócio, gestão estratégica, arquitetura modular e práticas ágeis para sustentar o ciclo de vida do software dentro desses ecossistemas. O sucesso de uma aplicação depende não apenas de sua qualidade interna, mas de sua capacidade de interoperar, evoluir e agregar valor ao ecossistema ao qual pertence.

Neste tópico, abordaremos desde o entendimento do negócio digital e a gestão do desenvolvimento até as arquiteturas orientadas a serviços, a customização e as práticas colaborativas que sustentam o desenvolvimento ágil e contínuo.

6.1 Compreendendo o ecossistema e o negócio digital

O início para o desenvolvimento de software em ecossistemas digitais é o entendimento do negócio e do ambiente no qual o sistema será inserido. Nos modelos tradicionais, o software atende a um conjunto fechado de requisitos de um cliente específico. Já os ecossistemas digitais operam como **redes abertas**, nas quais múltiplos atores, empresas, usuários, desenvolvedores, provedores de serviços e plataformas tecnológicas interagem, criando valor de forma contínua.

Observe a figura a seguir.

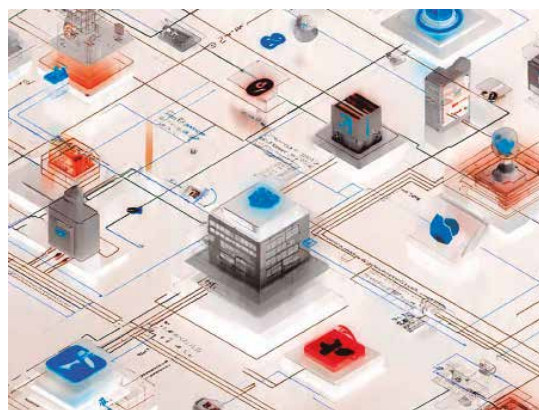


Figura 34 – Modelo de um ecossistema digital
(figura gerada pela ferramenta de inteligência artificial Meta IA)

Na figura apresentada, o ecossistema digital se baseia na interconectividade que identifica as relações de dependência, cooperação e competição e que moldam o contexto do software. São empregadas plataformas como Android, AWS, GitHub e Salesforce, as quais são exemplos emblemáticos, já que não apenas fornecem infraestrutura tecnológica, mas criam um ambiente de inovação compartilhada,

em que cada participante contribui para o crescimento coletivo. Nesses ambientes, o valor não está apenas no produto desenvolvido, mas na capacidade de integrar-se e gerar sinergia com outros serviços e aplicações.

O **negócio digital** em uma grande malha, conforme pode ser visto na figura 34, envolve a análise dos modelos de valor, fluxos de informação e necessidades dos usuários. A engenharia de software incorpora práticas de análise de negócios (business analysis), pensamento de design (design thinking) e modelagem do processo de negócios (business process modeling – BPM) para alinhar as soluções às estratégias corporativas e aos objetivos de mercado. O desenvolvedor deixa de ser apenas um executor técnico e passa a atuar como agente de transformação digital, capaz de compreender métricas de qualidade, oportunidades e impactos econômicos do software dentro do ecossistema.

Compreender o ecossistema e o negócio digital é estabelecer a base sobre a qual se estruturam todas as etapas subsequentes do projeto. Essa visão sistêmica permite que o software seja projetado não apenas para atender requisitos imediatos, mas para evoluir de forma sustentável, adaptando-se a novos parceiros, tecnologias emergentes e mudanças no comportamento dos usuários como características essenciais de um ecossistema digital bem-sucedido.

Em resumo, o ecossistema digital é formado por um conjunto de elementos interdependentes que trabalham juntos para criar, entregar, integrar e evoluir produtos e serviços digitais. Esses elementos não existem isoladamente: eles cooperam, competem e se influenciam, formando um ambiente dinâmico, vivo e em constante transformação.

Os principais componentes de um ecossistema digital moderno são:

- **Plataformas e infraestruturas tecnológicas:** é a base sobre a qual tudo funciona. Incluem:
 - sistemas operacionais (Android, iOS, Windows);
 - plataformas de nuvem (AWS, Azure, Google Cloud);
 - marketplaces e hubs de integração (Salesforce, Shopify).

Nessas plataformas são estabelecidos padrões, APIs, regras e protocolos que orientam a inovação.

- **Comunidades de desenvolvedores e parceiros:** são as pessoas e organizações que cocriam soluções dentro do ecossistema, como:
 - desenvolvedores independentes;
 - empresas terceiras;
 - startups;
 - fornecedores de serviços complementares.

Eles ampliam o ecossistema com apps, plugins, integrações e serviços.

- **Usuários e consumidores:** é o centro do ecossistema digital. Incluem:

- usuários finais;
- empresas clientes;
- organizações públicas ou privadas.

O comportamento dos usuários influencia diretamente a evolução das plataformas.

- **Serviços digitais, APIs e módulos interoperáveis:** conjunto de funcionalidades que possibilitam comunicação e composição entre sistemas, como:

- APIs abertas;
- microsserviços;
- web services;
- gateways e middlewares.

Eles garantem interoperabilidade, expansibilidade e integração.

- **Regras de governança e normas:** conjunto de diretrizes que mantêm o ecossistema digital coerente e funcional:

- políticas de uso e privacidade;
- licenças de software;
- standards (REST, OAuth, OpenAPI);
- boas práticas de comunidade.

A governança define como os atores podem participar e inovar.

- **Dados e fluxos de informação:** é o "combustível" do ecossistema digital como:

- dados de usuários;
- dados de operação;
- telemetria;

- analytics e métricas;
- bases de conhecimento.

Os dados orientam decisões, personalizações e melhorias contínuas.

- **Ferramentas de desenvolvimento e suporte:** um ecossistema digital é robusto e inúmeras ferramentas de desenvolvimento e suporte oferecem recursos para facilitar a criação e evolução de soluções:
 - SDKs e frameworks;
 - IDEs e repositórios;
 - DevOps, CI/CD, pipelines automatizados;
 - documentação, tutoriais e fóruns.

Quanto melhor o suporte, mais rápido o ecossistema cresce.

- **Aplicações, produtos e serviços criados dentro do ecossistema:** são os resultados tangíveis das interações:
 - apps móveis;
 - plataformas web;
 - plugins e extensões;
 - sistemas integrados;
 - soluções corporativas.

Eles demonstram o valor real do ecossistema para o mercado.

- **Comunidade de suporte, feedback e inovação:** é formada por:
 - fóruns;
 - grupos de usuários;
 - comunidades open source;
 - eventos e hackathons.

É a parte social do ecossistema digital, responsável por melhorar, testar e divulgar soluções.

- **Mecanismos de monetização e sustentabilidade:** permitem que o ecossistema digital se mantenha:
 - assinaturas, licenciamento, pay-per-use;
 - marketplaces que compartilham receita;
 - incentivos para desenvolvedores.

Sem sustentabilidade econômica, um ecossistema não prospera.

6.2 Gestão do desenvolvimento ágil em ecossistemas digitais

A gestão do desenvolvimento de software em ecossistemas digitais exige uma abordagem capaz de lidar com alta complexidade, interdependência entre sistemas, múltiplos atores e ritmos acelerados de inovação. O gerenciamento de projetos de software em ambientes conectados inclui planejamento, governança, integração de times e tomada de decisão orientada a dados.

A aplicação de métodos ágeis se faz essencial para garantir rapidez na entrega, adaptação contínua e alinhamento entre as diversas partes que compõem o ecossistema digital. A gestão ágil, quando aplicada a ambientes conectados, deixa de ser apenas um conjunto de práticas operacionais e transforma-se em um modelo estratégico de coordenação, que integra equipes, plataformas, usuários, dados e serviços distribuídos.

Nos ecossistemas digitais, a agilidade é impulsionada pelo próprio ambiente tecnológico, no qual aplicações consomem e oferecem serviços via APIs, dependem de arquiteturas distribuídas e evoluem de forma incremental. Com a colaboração entre múltiplos stakeholders em um ecossistema digital, o desenvolvimento não ocorre de maneira isolada: fornecedores, parceiros, desenvolvedores independentes e usuários finais influenciam diretamente o produto.

A gestão ágil atua como um mecanismo de alinhamento, permitindo que informações fluam com transparência e que decisões sejam tomadas com base em dados, métricas e validação contínua. Práticas como backlog compartilhado, definição clara de responsabilidades e cadências de sincronização entre equipes são determinantes para garantir coerência entre as diversas partes envolvidas.

No que se refere à evolução contínua do software em ambientes onde mudanças são frequentes, seja por atualizações de plataforma, requisitos regulatórios ou novas demandas de mercado, a capacidade de adaptar rapidamente arquiteturas, módulos e integrações torna-se indispensável. A agilidade, nesse sentido, não é apenas uma metodologia, mas uma competência organizacional que permite evoluir o software com o ecossistema digital, preservando compatibilidade, segurança e valor agregado.

Gerir o desenvolvimento ágil em ecossistemas digitais é construir uma abordagem integrada, colaborativa e orientada ao fluxo de valor, na qual o software é desenvolvido como parte de um

organismo maior. Essa visão garante que as soluções produzidas não apenas atendam às necessidades imediatas, mas sejam capazes de crescer, adaptar-se e permanecer relevantes em um ambiente digital em constante transformação.

6.3 Ecossistemas distribuídos e inteligentes

A crescente complexidade dos sistemas digitais tem impulsionado a evolução dos ecossistemas distribuídos e inteligentes, que combinam arquiteturas descentralizadas com capacidades de adaptação, automação e tomada de decisão. Esses ecossistemas são compostos por múltiplos serviços, dispositivos e agentes que operam de forma colaborativa, muitas vezes em tempo real, sobre infraestruturas escaláveis e heterogêneas. Tecnologias como computação em nuvem, microserviços, APIs, inteligência artificial e Internet das Coisas (IoT) convergem para formar ambientes dinâmicos, resilientes e orientados a dados.

6.3.1 Arquitetura distribuída

O desenvolvimento de software para internet evoluiu de aplicações monolíticas e centralizadas para arquiteturas distribuídas, capazes de operar em larga escala, atender múltiplos usuários simultaneamente e integrar-se a sistemas externos. Essa transformação foi impulsionada pela necessidade de interoperabilidade, disponibilidade contínua e flexibilidade para responder às mudanças nos ecossistemas digitais.



Lembrete

No controle da qualidade com SWEBOK®, das dezoito áreas de conhecimento (KAs), oito estão associadas diretamente ao desenvolvimento de arquiteturas distribuídas. São elas: arquitetura de software (software architecture), projeto de software (software design), construção de software (software construction), teste de software (software testing), qualidade de software (software quality), gerenciamento de engenharia de software (software engineering management), processo de engenharia de software (software engineering process) e segurança de software (software security).

Os sistemas distribuídos são muito parecidos com os sistemas tradicionais. Atuam como controladores de recurso para o hardware, permitindo que múltiplos usuários e aplicações compartilhem recursos como CPUs, memórias, periféricos, rede e dados de todos os tipos.

As categorias de multiprocessamento em sistemas distribuídos ocorrem de duas formas:

- **Computadores multiprocessados (multiprocessamento local):** trata-se de um único computador com múltiplos processadores (ou núcleos) que compartilham memória e recursos. Suas aplicações se baseiam em computação científica, processamento paralelo de alto desempenho,

inteligência artificial e simulações complexas. Suas principais características são: alta velocidade de comunicação entre processadores, memória compartilhada e gerência centralizada de recursos.

- **Sistemas distribuídos em rede (multiprocessamento distribuído):** em sistemas distribuídos, cada computador executa uma aplicação específica; integrados, formam um único ambiente operacional controlado pelo sistema distribuído. É um conjunto de computadores autônomos interligados por rede, operando como um sistema único. Suas aplicações se baseiam em sistemas corporativos e web, computação em nuvem, serviços baseados em microsserviços e APIs. Suas principais características são: comunicação via rede, execução distribuída e paralela, tolerância a falhas e escalabilidade horizontal.

Em sistemas distribuídos, como mostra a figura a seguir, o middleware é o principal componente e está localizado exatamente entre as camadas de aplicação e transporte de um sistema operacional de rede (SOR).

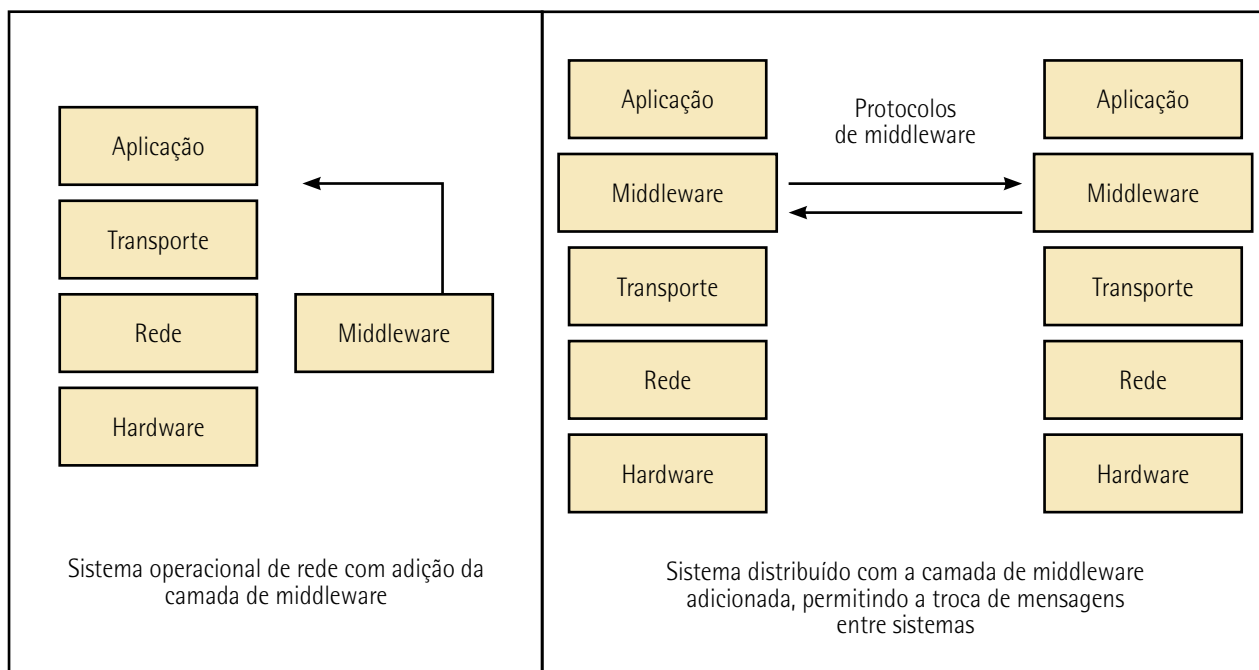


Figura 35 – Implementação da camada de rede middleware para um sistema distribuído

As aplicações atuais são concebidas como **conjuntos de serviços independentes**, conectados por interfaces bem definidas, que podem ser atualizados, ampliados ou substituídos sem comprometer o funcionamento global do sistema.

Uma característica fundamental dos sistemas distribuídos é que os usuários pensam o sistema como um único computador. Uma visão componentizada do diagrama de componentes/implantação, conforme visto na figura a seguir (figura 36), que mostra que sistemas operacionais distintos trocam mensagens e objetos pelo middleware.

O módulo do middleware responsável por essas operações é chamado de object request broker (ORB), que traduzido do inglês significa corretor de solicitação de objetos. Esse módulo é responsável pela conversão dinâmica dos objetos entre os sistemas operacionais. É uma solução de infraestrutura de TI, que para o usuário é imperceptível, exceto se houver alguma falha.

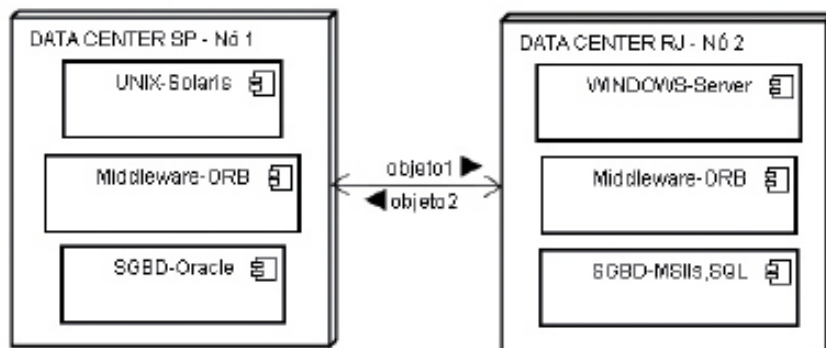


Figura 36 – Comunicação entre ambientes operacionais diferentes, com troca de objetos via middleware – object request broker (ORB)

Os principais padrões para computação distribuída que utilizam a tecnologia ORB são:

- Common Object Request Broker Architecture (CORBA);
- Distributed Component Object Model (DCOM);
- Java Remote Method Invocation (Java RMI).

Os dois padrões aceitos para todas as arquiteturas de objetos distribuídos são o CORBA e o DCOM. O padrão Java RMI é voltado para aplicações específicas.

O padrão CORBA, organizado pelo consórcio Object Management Group (OMG), que reúne mais de 500 empresas, destaca-se por sua flexibilidade, independência de linguagem e compatibilidade com múltiplos sistemas operacionais. Essas características fazem do CORBA uma das soluções mais adotadas pelo mercado para integração de sistemas distribuídos heterogêneos.

Uma forma prática de implementar o multiprocessamento distribuído é o uso de **cluster de computadores**, em que vários computadores (nós) trabalham juntos para executar tarefas como se fossem um único sistema. Cada nó pode executar uma parte da aplicação, e o sistema distribuído coordena a execução, o balanceamento de carga e a comunicação entre os nós.

Em um cluster como sistema distribuído, cada nó é um computador independente que executa aplicações específicas e colaborativas, sendo gerenciados por sistemas como Kubernetes, Hadoop ou Spark. São comuns em ambientes de negócios, data centers e aplicações web.

6.3.2 Service-oriented architecture (SOA)

A arquitetura orientada a serviços (SOA) é um marco nessa evolução ao introduzir o conceito de modularidade baseada em serviços reutilizáveis. Em uma SOA, funcionalidades específicas são expostas como **serviços interoperáveis**, que podem ser invocados por diferentes aplicações, plataformas ou parceiros de negócio. Essa abordagem permite que organizações construam soluções mais flexíveis, facilitando a integração entre sistemas heterogêneos e habilitando modelos de colaboração típicos de ecossistemas digitais, nos quais múltiplos agentes consomem e oferecem serviços simultaneamente.

SOA é um modelo de desenvolvimento que organiza sistemas em serviços independentes, reutilizáveis e interoperáveis, promovendo flexibilidade e escalabilidade em ambientes distribuídos. Sua aplicação é estratégica para atender aos requisitos de qualidade, suportar arquiteturas distribuídas e alinhar-se às práticas das metodologias ágeis.

A SOA contribui diretamente para atributos de **qualidade**, como:

- **Manutenibilidade:** serviços desacoplados facilitam atualizações e correções pontuais.
- **Reusabilidade:** componentes podem ser reaproveitados em diferentes contextos.
- **Escalabilidade:** serviços podem ser replicados ou distribuídos conforme a demanda.
- **Confiabilidade:** isolamento de falhas e monitoramento por serviço aumentando a robustez.

A SOA em **arquiteturas distribuídas** atua como base para:

- **Integração de sistemas heterogêneos:** via protocolos padronizados (SOAP, REST, gRPC).
- **Orquestração de serviços:** em ambientes como nuvem, clusters e microsserviços.
- **Gerência de comunicação:** entre serviços por barramentos (Enterprise Service Bus – ESB) ou mensageria. Ele permite que diferentes aplicações e serviços se comuniquem, por middleware, de forma desacoplada, escalável e interoperável, mesmo que estejam escritos em linguagens diferentes ou rodem em plataformas distintas.
- **Distribuição geográfica:** de componentes, mantendo consistência e desempenho.

A compatibilidade entre SOA e métodos ágeis ocorre por:

- **Entrega incremental:** serviços podem ser desenvolvidos e implantados em ciclos curtos.
- **Autonomia de equipes:** cada team pode focar em um serviço específico.
- **Testabilidade:** serviços isolados permitem testes automatizados e contínuos.
- **Adaptação rápida:** mudanças nos requisitos são absorvidas com menor impacto sistêmico.

A adoção de SOA em ecossistemas modernos permite que organizações construam soluções modulares, escaláveis e resilientes, alinhadas às exigências de mercado e às práticas de engenharia de software contemporânea.

Os **princípios da SOA** são baseados em diretrizes fundamentais que orientam o design e o desenvolvimento e integração de serviços em sistemas distribuídos. Eles garantem que os serviços sejam modulares, reutilizáveis, interoperáveis e alinhados aos objetivos de negócio.

Princípios fundamentais do SOA:

- **Serviços como unidades de funcionalidade:** cada serviço representa uma função de negócio bem definida, como processar pagamento ou consultar estoque.
- **Baixo acoplamento:** os serviços são independentes entre si, permitindo que mudanças em um não afetem os demais.
- **Alta coesão:** cada serviço é focado em uma única responsabilidade, o que facilita manutenção e reuso.
- **Interoperabilidade:** os serviços devem ser acessíveis por diferentes plataformas, linguagens e sistemas, geralmente via padrões abertos (por exemplo: XML, JSON, HTTP, SOAP e REST).
- **Reusabilidade:** serviços são projetados para serem reutilizados em múltiplos contextos, evitando duplicação de lógica.
- **Abstração:** a implementação interna do serviço é escondida do consumidor; o que importa é a interface pública.
- **Descoberta e localização:** serviços podem ser registrados e descobertos dinamicamente por meio de catálogos ou repositórios (por exemplo: UDDI).
- **Contratos bem definidos:** cada serviço expõe um contrato formal (por exemplo: WSDL, OpenAPI) que descreve como ele deve ser utilizado.
- **Autonomia:** os serviços controlam seus próprios recursos e lógica, podendo ser implantados e escalados de forma independente.
- **Composição:** serviços podem ser combinados para formar processos de negócio mais complexos, promovendo orquestração e flexibilidade.

Esses princípios fazem da SOA uma base sólida para novas arquiteturas, como microsserviços, e são compatíveis com práticas ágeis e ambientes de computação em nuvem.

A figura 37 a seguir apresenta uma representação estruturada da SOA, destacando seus principais componentes e a forma como se relacionam e se compõem de forma modular.

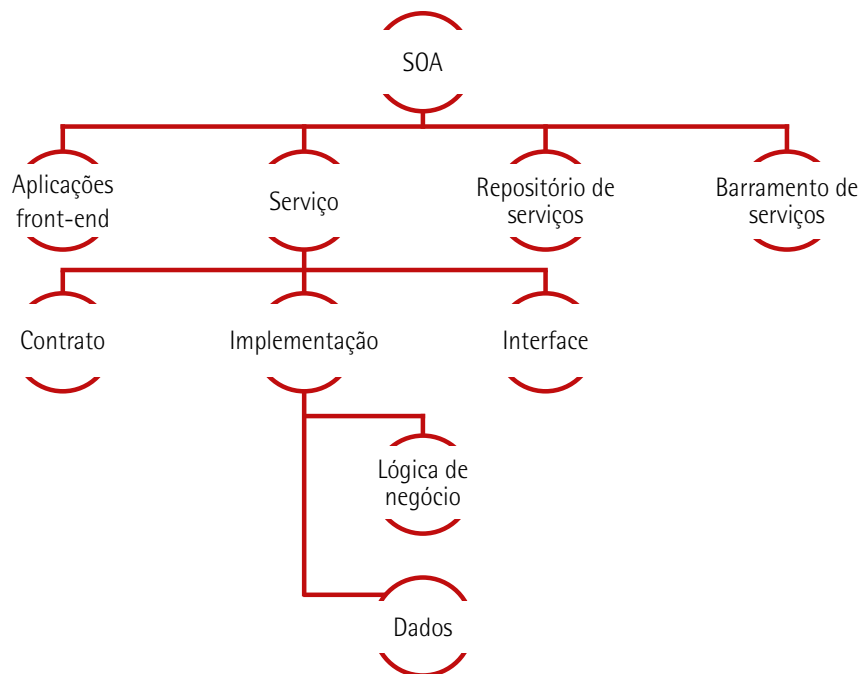


Figura 37 – Elementos de uma SOA

A arquitetura é organizada em camadas funcionais, que incluem aplicações front-end, contratos de serviço, implementação, interfaces, lógica de negócio, dados, repositórios e barramentos de integração.

6.3.3 Application programming interfaces (APIs)

Com a expansão da internet e o amadurecimento das tecnologias de nuvem, as SOAs deram lugar a arquiteturas mais granulares, como microsserviços, e modelos amplamente baseados em interfaces de programação de aplicações, ou application programming interfaces (APIs).

As APIs são as principais formas de exposição de funcionalidades na web, permitindo que aplicações interajam com plataformas, bancos de dados, dispositivos e serviços externos de maneira padronizada e segura. As APIs são elos que conectam empresas, desenvolvedores e usuários, possibilitando a criação de soluções inovadoras a partir da recombinação de serviços já existentes.

A convergência entre APIs e arquiteturas distribuídas constitui a base dos ecossistemas digitais, viabilizando soluções escaláveis, interoperáveis e centradas na experiência do usuário. As APIs funcionam como interfaces padronizadas de comunicação, permitindo que aplicações acessem funcionalidades, dados e serviços externos de forma segura e modular.

As arquiteturas distribuídas, além de promover interoperabilidade, respondem também às demandas de **escalabilidade e resiliência**. A distribuição de componentes entre servidores, containers e ambientes em nuvem permite que o sistema se adapte automaticamente ao volume de acessos, mitigando falhas e garantindo desempenho.

Nesse cenário, o desenvolvimento de software para internet requer uma visão arquitetural que considere não apenas a funcionalidade interna da aplicação, mas sua capacidade de coexistir e evoluir dentro do ecossistema digital.

Isso inclui cuidar do versionamento de APIs, manter contratos de interface estáveis e adotar padrões como REST.

- **REST (Representational State Transfer):** estilo arquitetural baseado em recursos acessados por URLs, utilizando protocolos HTTP como GET, POST, PUT e DELETE para operações, em que cada recurso tem um endpoint fixo, como /usuarios, /produtos. As respostas em geral são em JSON ou XML. É usado para implementar autenticação e autorização robustas como OAuth e monitorar o desempenho dos serviços expostos. A arquitetura deixa de ser um componente técnico isolado e passa a ser parte estratégica do modelo de negócio digital.
- **OAuth (Open Authorization):** é o protocolo de autorização que permite que um aplicativo acesse recursos de outro serviço em nome do usuário, sem expor suas credenciais. Muito usado em integrações com serviços como Google, Facebook, GitHub e outros, funciona com tokens de acesso temporários que representam permissões concedidas.

Um fluxo típico desse serviço é:

- 1) O usuário autoriza o aplicativo.
- 2) O serviço emite um token de acesso.
- 3) O aplicativo usa esse token para acessar os dados do usuário.

Exemplo prático de uso do OAuth: um app de calendário acessa seus eventos do Google Agenda sem precisar do seu login e senha, bastando ter o token autorizado.



Observação

As aplicações das APIs em uso no mundo real abrangem diversas APIs amplamente conhecidas que ilustram a aplicação prática em ecossistemas digitais. Serviços como Google Maps API e OpenWeather API viabilizam funcionalidades de geolocalização e dados em tempo real; APIs de pagamento, como Stripe, PayPal e Mercado Pago, permitem a integração segura de transações financeiras; enquanto plataformas em nuvem, como AWS, Azure e Google Cloud, disponibilizam APIs para provisão e gerenciamento de infraestrutura. Além disso, APIs de autenticação e comunicação, como OAuth, Auth0 e Twilio, demonstram como essas interfaces são essenciais para a integração, escalabilidade e automação de soluções de software em ambientes distribuídos.

Exemplo de aplicação

Estudo de caso: inclusão da API de clima

Este caso se refere à OpenWeather®, uma chamada de API para receber acesso aos vários dados, como atual clima e previsões, dados meteorológicos, visão geral do tempo e outros recursos.

Veja o que o código abaixo faz:

- conecta-se à API REST da OpenWeatherMap;
- envia uma requisição HTTP GET com parâmetros como cidade e chave de acesso;
- recebe uma resposta em JSON e extrai dados relevantes;
- exibe a temperatura e a descrição do clima em português.

Código para implementação da API de clima em software:

```
import requests
# Dados da API
API_KEY = 'sua_chave_api_aqui' # Substitua pela sua chave da OpenWeatherMap
cidade = 'São Paulo'
url = f'https://api.openweathermap.org/data/2.5/weather?q={cidade}&appid={API_KEY}&units=metric&lang=pt_br'
# Requisição à API
resposta = requests.get(url)
# Verificação e exibição dos dados
if resposta.status_code == 200:
    dados = resposta.json()
    temperatura = dados['main']['temp']
    descricao = dados['weather'][0]['description']
    print(f'Temperatura atual em {cidade}: {temperatura}°C - {descricao}')
else:
    print(f'Erro ao acessar API: {resposta.status_code}')
```

6.4 Customização de serviços em ecossistemas digitais

A **customização de serviços em ecossistemas digitais** representa uma estratégia essencial para atender às demandas específicas de usuários, aplicações e contextos operacionais. Em ambientes distribuídos e dinâmicos, como os que compõem arquiteturas baseadas em microsserviços, a capacidade de adaptar funcionalidades, interfaces e comportamentos de serviços é um diferencial competitivo e técnico.

Essa **customização ocorre em múltiplas camadas**, que vão desde a parametrização de APIs e configuração de endpoints (local digital onde se hospeda a API) até a personalização da lógica de negócio e da experiência do usuário. Essas ferramentas servem para gerenciar a complexidade de aplicações distribuídas, normalmente baseadas em microsserviços, automatizando tarefas de rede, segurança e infraestrutura. Elas permitem aos desenvolvedores focar apenas na regra de negócio (o código da aplicação), enquanto as ferramentas cuidam das responsabilidades operacionais comuns.

Essas ferramentas incluem:

- **API gateways:** componente de software que atua como único ponto de entrada para um conjunto de serviços backend (frequentemente microsserviços).
- **Service meshes (malha de serviços):** é a camada de infraestrutura que gerencia a comunicação entre serviços de uma aplicação distribuída.
- **Plataformas de orquestração:** atuam como um maestro. São ferramentas de software projetadas para automatizar e coordenar o gerenciamento de múltiplos sistemas, serviços e tarefas em um ambiente de tecnologia da informação (TI).

Por exemplo, o Kubernetes (K8s) é uma plataforma open source para orquestração de contêineres que automatiza a implantação, o escalonamento e o gerenciamento de aplicações containerizadas. Ele permite que serviços sejam ajustados em tempo real, com controle de versionamento, roteamento inteligente e políticas de segurança específicas no domínio da aplicação, e oferece alta disponibilidade e autocorreção em clusters de servidores. É o padrão de mercado para rodar microsserviços e aplicações distribuídas na nuvem.

Observe a figura a seguir. Os nodes 1 e 2 (nós) hospedam os pods (unidades lógicas) que são os componentes da carga de trabalho da aplicação. O control plane (plano de controle), o cérebro do cluster, gerencia os worker nodes (nós trabalhadores) e os pods no cluster. Em ambientes de produção, o control plane geralmente executa em múltiplos computadores e um cluster geralmente executa múltiplos nodes, fornecendo tolerância a falhas e alta disponibilidade.

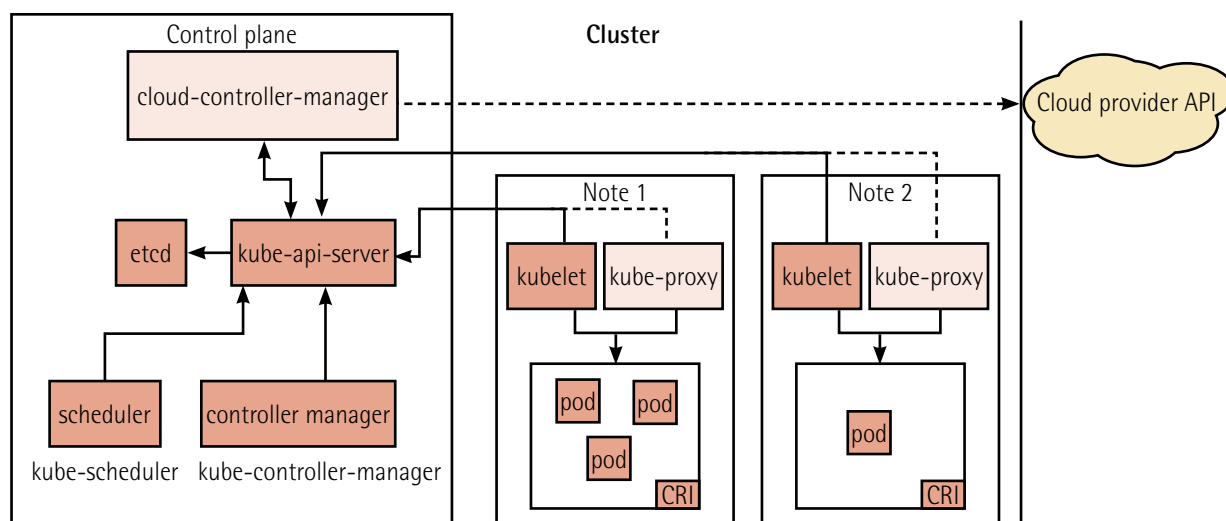


Figura 38 – Componentes do cluster Kubernetes



Saiba mais

Acesse o site a seguir:

ARQUITETURA do cluster: os conceitos arquiteturais por trás do Kubernetes. *Kubernetes*, 2025. Disponível em: <https://tinyurl.com/2edbr9c9>. Acesso em 23 dez. 2025.

A **customização nas metodologias ágeis** é viabilizada por ciclos curtos de entrega (do inglês, delivery), testes automatizados e feedback contínuo. Equipes multidisciplinares podem desenvolver e ajustar serviços de forma incremental, promovendo resiliência, escalabilidade e alinhamento com os objetivos de negócio. A abordagem orientada a produto, comum em ecossistemas digitais, favorece a criação de serviços modulares que podem ser recombinaados e evoluídos conforme as necessidades emergentes.

Além disso, o uso de infraestrutura como código (IaC), que é uma abordagem que permite o monitoramento contínuo e design centrado no usuário, reforça a capacidade de adaptação dos serviços, garantindo que o ecossistema digital permaneça responsivo, seguro e eficiente frente às mudanças tecnológicas e de mercado.

Em vez de procedimentos manuais, a IaC permite provisionar, configurar e gerenciar infraestrutura de TI usando arquivos de código. Em vez de criar servidores, redes, bancos de dados e serviços clicando em telas de painel, você descreve tudo em código, e a infraestrutura é criada automaticamente a partir do código.

Exemplo: criação de uma máquina virtual na AWS.

```
resource "aws_instance" "web" {  
  ami = "ami-0abcd1234"  
  instance_type = "t2.micro"  
}
```

A customização de serviços em ecossistemas digitais é aplicada de forma prática por meio de uma combinação de arquitetura distribuída, metodologias ágeis e ferramentas de automação e orquestração. Esse guia prático pode ser dividido nas etapas a seguir:

- 1) Modelagem de serviços modulares:** defina serviços com escopo claro em que cada serviço represente uma funcionalidade de negócio específica. Use design orientado a domínio (DDD) para alinhar os serviços às necessidades reais dos usuários e processos. O DDD permite criar um modelo mental dos especialistas do negócio.
- 2) Exposição via APIs padronizadas:** utilize REST para apresentar as funcionalidades de forma interoperável, que são funcionalidades que se comunicam e operam com outros sistemas.
- 3) Configuração e parametrização dinâmica:** implemente variáveis de ambiente e arquivos de configuração externos, que é a separação das informações de configuração do código-fonte, para adaptar o comportamento dos serviços sem alterar o código. Use chaves de funcionalidades (do inglês feature toggles) para ativar ou desativar funcionalidades, promovendo uma alternância de funcionalidades, conforme o domínio ou perfil do usuário.
- 4) Orquestração e roteamento inteligente:** adote uma service mesh (malha de serviços) para controlar o tráfego entre serviços, aplicar políticas de segurança e realizar balanceamento de carga, e uma API Gateway para personalizar rotas, autenticação e limitação de acesso por cliente ou aplicação.
- 5) Entrega contínua e iterativa:** configure pipelines de CI/CD com ferramentas como GitHub Actions ou GitLab CI para implantar serviços rapidamente, contêineres como a ferramenta Docker, que serve para empacotar, distribuir e executar aplicações dentro dos contêineres e orquestradores (Kubernetes) para escalar e adaptar os serviços conforme demanda.
- 6) Monitoramento e feedback contínuo:** implemente mecanismos visuais para um monitoramento avançado como Prometheus (com Grafana) para acompanhar o desempenho e uso dos serviços. Colete feedback dos usuários e métricas de uso para ajustar os serviços de forma ágil. O Prometheus serve para coleta e armazenamento de métricas, e o Grafana, que não coleta dados, permite visualizá-los em dashboards.
- 7) Personalização por perfil ou contexto:** integre sistemas de autenticação e autorização como o OAuth2 (protocolo de autorização) e o JWT (JSON Web Token), para validação digital de mensagens e funcionalidades, de acordo com o perfil do usuário. Os dados contextuais permitem construir experiências personalizadas como localização, dispositivo, histórico e outros. Por exemplo: entrar com Google e permitir que o app acesse seus arquivos do Dropbox e permissões da API (GitHub, Google Cloud e outros).

6.5 Métodos, ferramentas e técnicas (MF&T): desenvolvimento ágil em ecossistemas digitais

O funcionamento de um ecossistema digital lida com alta complexidade e volume de mudanças. A agilidade precisa coexistir com a estabilidade. Alguns exemplos do emprego de MF&T são aplicados no e-commerce de grande porte, serviços financeiros, plataformas de streaming e grandes empresas globais de tecnologia.

A agilidade é indispensável para prosperar na economia digital, e este tópico explora o desenvolvimento ágil aplicado e adaptado para operar dentro de complexos ecossistemas digitais. Da teoria à prática, frameworks de grande escala (SAFe®), cultura de automação de (DevOps), comunicação transparente de API gateways e service meshes, e a escalabilidade de plataformas de orquestração (Kubernetes) atuam em conjunto. O foco é garantir a entrega contínua e rápida de valor, mantendo a estabilidade e a qualidade em sistemas distribuídos e interconectados.

Métodos ágeis aplicados

A agilidade favorece a interoperabilidade entre sistemas, a escalabilidade arquitetural e a rápida incorporação de novas tecnologias ou componentes que surgem no ambiente digital. Os métodos ágeis estruturam serviços para o desenvolvimento de soluções em ecossistemas digitais, caracterizados por alta conectividade, dinamismo e evolução contínua.

Os produtos e serviços digitais precisam responder rapidamente a mudanças de mercado, novos requisitos dos usuários e integrações cada vez mais complexas entre plataformas, APIs, microsserviços e aplicações distribuídas. A agilidade não é apenas um modelo de gestão, mas uma competência estratégica para sustentar inovação, reduzir riscos e manter competitividade.

O quadro a seguir apresenta as principais metodologias ágeis aplicadas em ecossistemas digitais:

Quadro 17 – Metodologias ágeis aplicadas em ecossistemas digitais e microsserviços

Método	Aplicação no ecossistema digital
Scrum	Organização de equipes em sprints curtos para entregar incrementos de valor contínuos. Ideal para coordenação entre squads (em português, esquadrão), modelo organizacional usado em metodologias ágeis que define equipes autônomas para o desenvolvimento de diferentes serviços
Kanban	Visualização de quadro ou rede de cartões do fluxo de trabalho e gestão de tarefas em tempo real. Útil para times que lidam com demandas contínuas e manutenção de serviços
SAFe®	Framework de conhecimento e práticas para escalabilidade e implementação de conhecimentos e práticas do ágil em organizações com múltiplos times e produtos interdependentes
DevOps	É o conjunto de práticas culturais e técnicas para integrar desenvolvimento e operações que garantem entrega contínua, automação de infraestrutura e monitoramento constante

Ferramentas populares

Os ecossistemas digitais são sustentados por um conjunto crescente de ferramentas que possibilitam colaboração, automação, integração e operação contínua de serviços. Em ambientes altamente conectados compostos por APIs, microsserviços, plataformas em nuvem e aplicações distribuídas desempenham papel estratégico, constituindo uma base técnica que permite às organizações desenvolver, implantar e monitorar soluções com bom desempenho e confiabilidade.

São ferramentas de versionamento utilizadas para rastreamento e estruturação de equipes de desenvolvimento, integração e entregas contínuas de produtos de software, automatização de testes, builds e implantação, soluções containerizadas e orquestradas, ferramentas de monitoramento e análise que permitem melhor visibilidade operacional de um ecossistema digital.

O quadro a seguir destaca os principais serviços e emprego de ferramentas, conhecidas no mercado, utilizadas em ecossistemas digitais. O uso adequado das ferramentas de software fortalece a governança tecnológica e permite que produtos digitais se mantenham competitivos em um cenário marcado por mudanças rápidas e complexidade crescente.

Quadro 18 – Metodologias ágeis aplicadas em ecossistemas digitais e microsserviços

Categoria	Ferramenta	Finalidade
Gerenciamento ágil	Trello, Jira, Azure DevOps, ClickUp	Planejamento de sprints, backlog, tarefas e acompanhamento de progresso
Integração contínua/entrega contínua (CI/CD)	GitHub Actions, GitLab CI, Jenkins, CircleCI	Automatização de builds (conversão de código-fonte em software pronto para uso), testes e deploys (implantação) em ambientes distribuídos
Containers e orquestração	Docker, Kubernetes	Empacotamento e escalonamento de microsserviços em ambientes de nuvem
Monitoramento e observabilidade	Prometheus, Grafana, ELK Stack, New Relic	Acompanhamento de desempenho, logs e métricas de serviços distribuídos
APIs e integração	Postman, Swagger/OpenAPI, Kong, Apigee	Design, documentação, testes e gerenciamento de APIs em ecossistemas abertos

Técnicas práticas

As técnicas representam o conjunto de métodos operacionais utilizados no dia a dia do desenvolvimento, operação e evolução de produtos digitais, servindo como ponte entre princípios teóricos e a execução efetiva. Em ambientes de alta conectividade, escalabilidade elástica e integração constante entre serviços, tais práticas tornam-se vitais para as operações em ecossistemas digitais.

A aplicação estruturada dessas técnicas fortalece a maturidade digital das organizações e permite que produtos e serviços evoluam com rapidez, segurança e qualidade, respondendo de forma eficiente às demandas do mercado. As técnicas práticas, listadas a seguir, formam a estrutura que sustentam as operações em ecossistemas digitais cada vez mais dinâmicos e orientados a dados. As principais técnicas são:

- **Desenvolvimento orientado a testes (TDD):** garante qualidade desde o início, com testes automatizados para cada funcionalidade.
- **Integração contínua (CI):** código integrado ao repositório, com testes automáticos para detectar falhas rapidamente.
- **Entrega contínua (CD):** automatiza a entrega de software em produção com segurança e frequência.
- **Infraestrutura como código (IaC):** provisionamento automatizado de ambientes operacionais.
- **Design de APIs First:** planejamento e documentação de APIs antes da implementação, promovendo alinhamento entre equipes.
- **Arquitetura de microsserviços:** divisão do sistema em serviços independentes, facilitando a escalabilidade e a entrega incremental. A integração das metodologias ágeis com a arquitetura de microsserviços formam uma sinergia natural. Enquanto o ágil fornece o processo de feedbacks e entrega incremental, os microsserviços fornecem a estrutura técnica, independente e modular para mudanças rápidas, escaláveis e viáveis.
- **Pair programming (programação em pares) e code review (revisão de código):** pair programming envolve dois desenvolvedores trabalhando juntos em um único código, e code review consiste na inspeção sistemática do código por outros desenvolvedores para identificação de defeitos. Essas técnicas orientam a melhoria da qualidade do código e compartilhamento de conhecimento entre desenvolvedores.
- **MVP (produto mínimo viável):** lançamento rápido de versões básicas para validação com usuários reais.

Os benefícios do desenvolvimento ágil em ecossistemas digitais são inúmeros, dos quais se destacam:

- escalabilidade horizontal com microsserviços e containers;
- interoperabilidade via APIs padronizadas;
- resiliência e tolerância a falhas com arquiteturas distribuídas;
- adaptação rápida às mudanças com ciclos curtos de entrega;
- colaboração entre equipes multidisciplinares com visão orientada a produto.



Resumo

Nesta unidade, vimos a evolução do projeto e a construção de software a partir de uma visão integrada entre gestão, engenharia e arquitetura de sistemas. Inicialmente, destaca-se a importância do gerenciamento estruturado de projetos de software, que envolve planejamento, execução, monitoramento e encerramento de forma alinhada a prazos, custos, requisitos e qualidade. Dessa forma, o gerente de projetos desempenha um papel-chave na coordenação de equipes, na comunicação e no controle dos processos envolvidos.

A gestão tradicional, amplamente difundida pelo guia PMBOK®, é apresentada como um referencial que organiza o trabalho em grupos de processos e áreas de conhecimento, cobrindo desde a gestão do escopo e do cronograma até riscos, comunicações e partes interessadas. Esse modelo convive com abordagens adaptativas e metodologias ágeis, que oferecem flexibilidade, entregas incrementais e foco no valor gerado ao cliente. As práticas como Scrum, XP, Kanban, Lean e outras mais no contexto ágil são integradas ao ambiente de projetos, permitindo respostas rápidas às mudanças e maior colaboração entre equipes.

Em complemento à gestão, o guia SWEBOK® apresenta o corpo de conhecimento que das práticas técnicas da engenharia de software, abrangendo desde requisitos, design e construção até teste, manutenção e gestão da configuração, além de destacar a relevância de padrões internacionais como IEEE, ISO e CMMI® na padronização de processos e na melhoria da qualidade.

A arquitetura de sistemas surge como o elo entre os elementos de gestão e de engenharia. Foram discutidos seus tipos, componentes e impactos na qualidade, mostrando como as arquiteturas influenciam desempenho, segurança, escalabilidade e manutenção. Arquiteturas como monolíticas, distribuídas, orientadas a serviços e em camadas são avaliadas de forma comparativa, em que se destaca a importância do planejamento arquitetural para o sucesso do projeto.

Com a transformação digital, a engenharia de software passa a operar em ecossistemas dinâmicos e interconectados. Sendo o ecossistema digital, interpretado como um ambiente composto por plataformas, usuários, dados, serviços e tecnologias que se integram e se relacionam de forma contínua, o software assume um papel estratégico para inovação e criação de valor.

A gestão ágil se faz como um elemento central dos ecossistemas digitais, permitindo ciclos curtos, adaptação contínua, uso intensivo de métricas e colaboração entre equipes multidisciplinares. Os sistemas distribuídos, a computação em nuvem e os microsserviços compõem a base tecnológica desses ambientes, exigindo novos modelos de orquestração, comunicação e resiliência.

As arquiteturas distribuídas permitem a execução de componentes em diferentes nós, favorecendo escalabilidade e tolerância a falhas. A arquitetura orientada a serviços (SOA) reforça os princípios de reutilização, governança e integração entre serviços independentes. As APIs são essenciais nesse cenário, funcionando como canal de comunicação entre sistemas, ampliando a interoperabilidade e permitindo a criação de serviços combinados e inovadores.

Os serviços são personalizados e surgem como características primordiais dos ecossistemas digitais, possibilitando a coleta e análise de dados, a segmentação de usuários e a adaptação dinâmica de funcionalidades. Por fim, são destacados métodos, ferramentas e técnicas que apoiam o desenvolvimento ágil em ecossistemas digitais, como integração e entrega contínuas (CI/CD), automação dos processos e métodos, versionamento, testes contínuos, codificação, monitoramento e acompanhamento de serviços com práticas de DevSecOps (desenvolvimento + segurança + operações).

Esses elementos formam um conjunto integrado que permite construir, manter e evoluir soluções em ecossistemas digitais de maneira rápida, segura e eficiente, alinhada com as demandas de ecossistemas distribuídos e inteligentes.



Exercícios

Questão 1. Uma equipe está conduzindo um projeto ágil para desenvolver um sistema web. Na reunião de planejamento, o gerente pede que o processo de desenvolvimento seja delimitado com clareza, definindo o que entra no processo (inputs), o que acontece durante a execução (processo) e o que deve sair ao final (outputs), de modo que seja possível monitorar e controlar o andamento por marcos (milestones). Qual alternativa descreve corretamente essa delimitação em um contexto ágil?

- A) Input: histórias de usuário e backlog priorizado, recursos humanos e tecnológicos, infraestrutura de desenvolvimento e ferramentas de integração/automação. Processo: sprints, integração e entrega contínuas, testes automatizados, revisões de código e feedback do cliente. Output: entrega validada conforme critérios de aceitação, com medições e registros que alimentam melhoria contínua e novos ciclos.
- B) Input: entregas já validadas do produto. Processo: apenas registro de métricas ao fim do mês. Output: backlog e histórias de usuário, definidos somente após a execução.
- C) Input: escolha de um framework por preferência pessoal. Processo: apenas reuniões de status sem transformação de requisitos. Output: expectativa de valor sem critérios de aceitação nem validação.
- D) Input: arquitetura final detalhada e imutável. Processo: implementação sem testes nem revisão de código para aumentar velocidade. Output: entrega sem medições, verificação ou registro, para evitar "burocracia".
- E) Input: somente estimativas de custo e de tempo. Processo: atividades gerenciais desconectadas das tarefas técnicas. Output: documentos de planejamento, sem incremento funcional.

Resposta correta: alternativa A.

Análise das alternativas

A) Alternativa correta.

Justificativa: a alternativa descreve corretamente input, processo e output. Os inputs incluem histórias de usuário/backlog, recursos, infraestrutura e ferramentas. O processo descreve a transformação contínua dos requisitos em incrementos via sprints, inclui CI/CD e tem testes automatizados, revisão de código e feedback. O output envolve a entrega validada com medições e registros, favorecendo o controle e a melhoria contínua.

B) Alternativa incorreta.

Justificativa: a alternativa inverte a lógica: trata a entrega validada como entrada e desloca o backlog/histórias para a saída, contrariando a definição de inputs como os requisitos e os recursos necessários para iniciar o ciclo.

C) Alternativa incorreta.

Justificativa: a alternativa reduz o input à preferência pessoal, esvazia o processo (sem transformação estruturada) e não define o output com validação por critérios de aceitação.

D) Alternativa incorreta.

Justificativa: a alternativa elimina práticas descritas como parte do processo (os testes automatizados e as revisões) e ignora a necessidade de medições/verificações/registro no encerramento para comparar os resultados com os objetivos.

E) Alternativa incorreta.

Justificativa: a alternativa restringe indevidamente os inputs e separa a gestão da execução técnica.

Questão 2. Uma empresa está definindo a arquitetura de um sistema que integra múltiplos módulos e precisa atender aos requisitos de qualidade (por exemplo, desempenho, escalabilidade e segurança). O gerente conduz o projeto com base em práticas de gerenciamento (escopo, qualidade e riscos), enquanto a equipe técnica busca referência para conceber e documentar a arquitetura e seus elementos (componentes, interfaces e protocolos). Qual conduta está mais aderente à visão integrada entre gerenciamento do projeto e corpo de conhecimento técnico?

A) Manter a arquitetura fora do escopo para evitar as restrições, deixando as decisões arquiteturais somente para o final, sem o registro de riscos.

B) Tratar a arquitetura como um desenho ilustrativo para uma apresentação, sem ter relação com a verificação e a validação dos atributos de qualidade.

C) Definir a arquitetura como um entregável do projeto no escopo. Usar a arquitetura para integrar o trabalho entre as equipes. Verificar e validar os atributos de qualidade definidos na arquitetura. Registrar e analisar os riscos decorrentes de inadequação ou de falta de documentação arquitetural, ao mesmo tempo em que a equipe descreve e documenta os elementos como módulos/serviços, como interfaces e como protocolos.

D) Selecionar uma arquitetura "de mercado" sem análise de requisitos, uma vez que os padrões internacionais dispensam a avaliação do contexto do projeto.

E) Evitar a documentação arquitetural para permitir as mudanças constantes, mesmo quando há requisitos explícitos de desempenho e escalabilidade.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: a alternativa contraria a ideia de arquitetura como entregável relevante para o planejamento e para a execução, além de ignorar o tratamento de riscos associados à arquitetura.

B) Alternativa incorreta.

Justificativa: a alternativa desconsidera que as decisões arquiteturais influenciam diretamente os atributos de qualidade e que esses atributos podem ser verificados e validados por processos de qualidade do gerenciamento.

C) Alternativa correta.

Justificativa: a alternativa reúne os seguintes pontos centrais: a arquitetura atua como uma ponte entre o planejamento/gestão e a execução técnica; no gerenciamento, impacta a integração, o escopo, a qualidade e os riscos; no corpo técnico, envolve a concepção e a documentação da estrutura (seus componentes/serviços, suas interfaces e seus protocolos), influenciando atributos como o desempenho, a escalabilidade, a segurança e a manutenibilidade.

D) Alternativa incorreta.

Justificativa: a decisão arquitetural depende do contexto e dos requisitos. A alternativa elimina a etapa de análise que fundamenta a arquitetura.

E) Alternativa incorreta.

Justificativa: a ausência de documentação é apresentada como risco para o projeto, especialmente quando há requisitos de qualidade que dependem de decisões arquiteturais explícitas.